

FUNCTIONAL GENOMICS · LECTURE 02

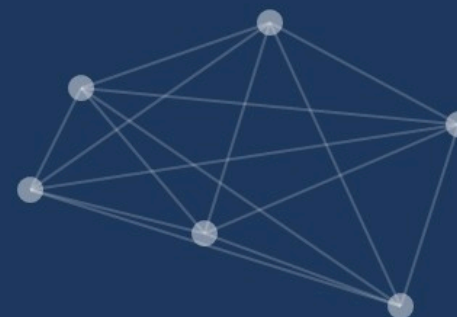
数据可视化

把数据变成别人一眼就懂的图

功能基因组学 · 研究生课程

南方医科大学 基础医学院 生物信息学系

余光创



1 / 96

OUTLINE

今天要解决的问题

- 01 为什么要用代码画图，而不是 Excel
- 02 R 的三套图形系统：base / lattice / ggplot2
- 03 图形语法：一张图由哪几个部件组成
- 04 常用 geom、scale、coord、theme 怎么选
- 05 复杂图：拼图的本质是「数据关联」
- 06 生态与趣味：从 ggtree 到 geom_cake
- 07 上机练习
- 08 收尾：常踩的坑与学习路径

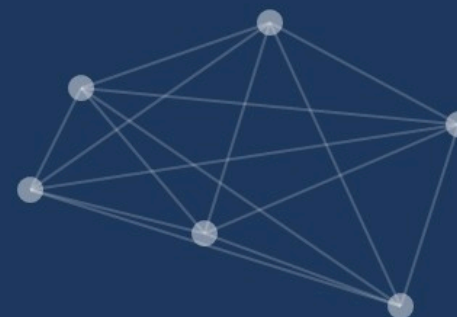
2 / 96

PART 01

为什么要用代码画图

先想清楚这个问题，后面才学得进去

- 一张图说服了一个时代
- 工具的全景与边界
- Excel 能做到什么，做不到什么
- 为什么是 R



3 / 96

A FAMOUS QUOTE



**Let us not kid ourselves:
the most widely used piece of
software for statistics is Excel.**

— Brian D. Ripley

牛津大学统计学家、R Core 成员

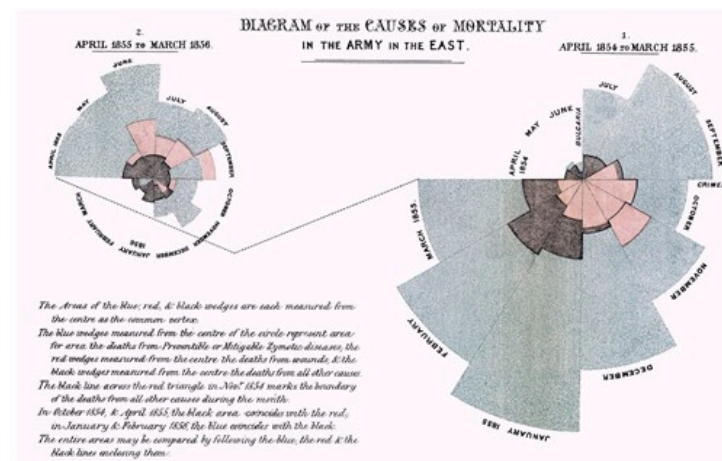
4 / 96

DATA VISUALIZATION IS EVERYWHERE

1857：一张图救了多少人

南丁格尔用「玫瑰图」说服了女王

- 克里米亚战争中，死于疾病的士兵远多于战死
- 她把死亡原因按月画成堆叠扇形图
- 一眼就能看出：问题不在战场，在卫生条件
- 结果：军队医疗改革，近代护理事业由此开启



南丁格尔玫瑰图 (Nightingale Rose Diagram)

5 / 96

LANDSCAPE

生物医学里常见的可视化工具

它们都很好，但适用场景不一样

Microsoft Excel

上手最快。适合小数据、一次性探索。自动化与可复现性弱。

GraphPad Prism

实验生物学常用。统计与作图一体，图形规范，但自定义能力有限。

Origin / SigmaPlot

理工科传统强项。精细排版，但脚本化与批量处理偏弱。

IBM SPSS

菜单式统计。适合问卷与社会调查，图形偏保守。

R + ggplot2

图形语法 + 完整编程能力。可复现、可批量、可扩展。

Python + matplotlib

生态强大，与深度学习无缝衔接。语法比 ggplot2 更啰嗦。

6 / 96

WHY WE NEED TO CODE

Excel 的坑，不在「能不能画」，而在「能不能重来」

数据一改，前面所有手工操作都要重做一遍

不可复现

拖拽、筛选、手动调色，这些动作没有留下任何记录

不可审计

审稿人问「这个点怎么来的」，你只能凭记忆复述

难以扩展

10 个基因可以手画，2000 个基因只能靠代码

容易出错

复制粘贴时错行、公式引用范围漂移，肉眼很难发现

协作困难

别人拿到你的文件，看不出每一步做了什么

不可版本化

文件改乱了只能靠「最终版2_改好的」来区分

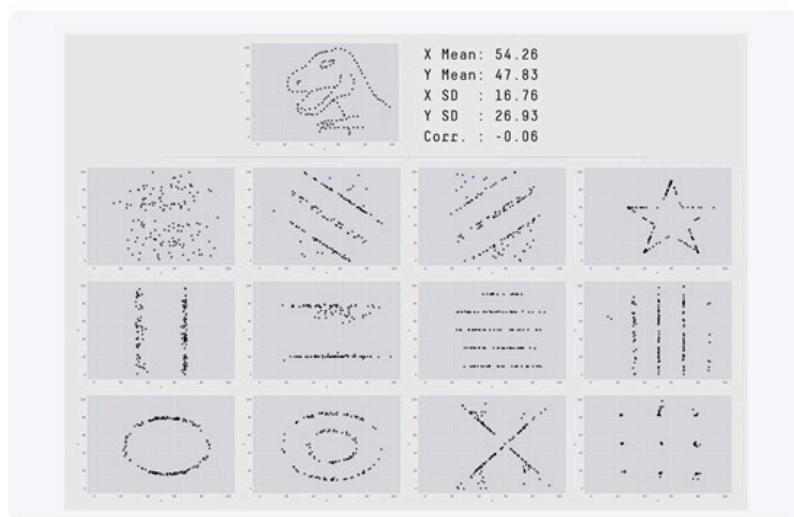
代码的价值不是画得更漂亮，而是每一步都写得下来、重跑得出来

7 / 96

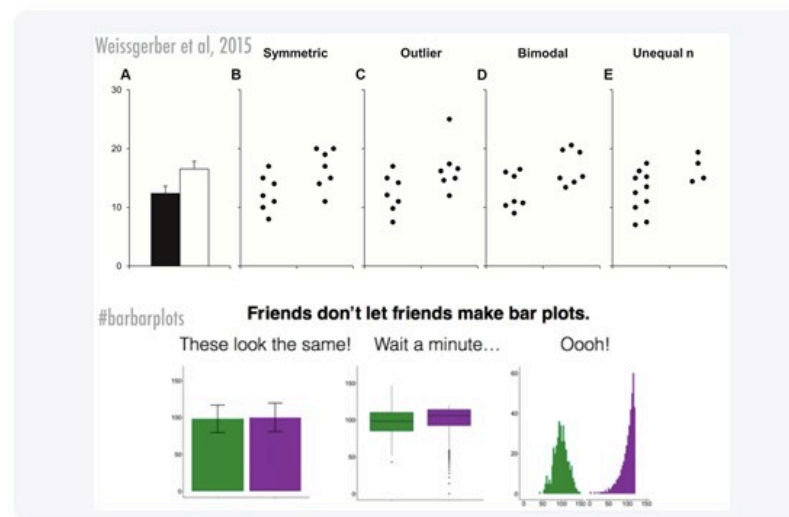
WHY WE NEED TO CODE

均值和标准差相同，不代表数据相同

左：13 张图共享同一套 mean/SD/相关系数；右：均值±误差线相同，分布却完全不同



Datasaurus dozen: 13 张图的 mean、SD、corr 完全一致

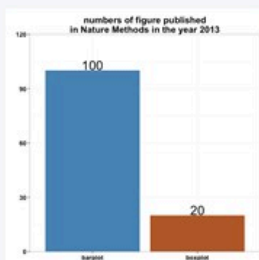


Weissgerber et al. 2015: 柱图 + 误差线相同，底下的数据完全不同

WHY WE NEED TO CODE

从「为什么不用 boxplot」到用 R 自己探索

吐槽之后是工具，工具之后是能力：R 让我们用代码，自己完成可视化探索



① 2013 年一整年：柱状图 100 张，boxplot 只有 20 张



② 该刊自己下场吐槽：Why not use boxplot?

BoxPlotR: a web tool for generation of box plots

Michaela Spitzer, Jan Wildenhain, Juri Rappsilber & Mike Tyers

Affiliations | Corresponding authors

Nature Methods 11, 121–122 (2014) | doi:10.1038/nmeth.2811

Published online 30 January 2014

③ 回应吐槽的是工具：BoxPlotR，Nature Methods 2014 年 1 月



④ 但工具之外还有更根本的一步：别在 Excel 里点，用 R 写下来

9 / 96

WHY R

R: 为统计计算与图形而生的语言

1993 年诞生，三十多年仍是生物信息学的主场

1993

诞生于新西兰奥克兰

Robert Gentleman 与 Ross Ihaka
实现，是 S 语言的开源实现

23k+

CRAN 上的包

覆盖统计、绘图、机器学习、
生物信息等几乎所有方向

2.3k+

Bioconductor 软件包

高通量组学数据分析的事实标
准仓库

免费

开源、跨平台

Windows / macOS / Linux 全
支持，没有任何授权费用

R 的强项：统计建模 + 数据操作 + 出版级图形，三者在一个语言里闭环。

10 / 96

TWO VOICES



S has forever altered the way people analyse, visualise and manipulate data.

— John Chambers

S 语言的创造者之一，贝尔实验室

11 / 96

TWO VOICES



**R is the most powerful and flexible
statistical programming
language in the world.**

— Norman Nie

SPSS 联合创始人

12 / 96

ROADMAP

今天的路线

从「为什么」到「怎么做」，再到「怎么做得像作品」

1 动机

为什么代码画图

2 全景

R 的三套图形系统

3 语法

一张图的组成部件

4 上手

常用 geom / scale / theme

5 进阶

拼图与数据关联

6 趣味

生态与彩蛋

学完你至少能：看懂任何一张 ggplot2 图，并把它改造成自己想要的形状

13 / 96

A TABLE OF NUMBERS

120 个数字，你能看出什么？

10 个克隆系的适合度，每 10 天测一次，共 12 个时间点（模拟数据）

克隆系	0	10	20	30	40	50	60	70	80	90	100	110	天
C1	3.04	2.39	2.84	3.20	3.29	3.49	4.11	3.95	4.04	4.56	3.93	4.42	
C2	3.04	2.86	3.66	3.57	3.44	3.79	3.98	4.27	4.16	3.97	3.91	2.86	
C3	2.76	2.87	3.42	3.71	3.53	3.88	3.76	4.13	3.79	3.46	3.02	2.58	
C4	2.68	2.56	3.26	3.74	4.23	3.80	3.88	3.42	3.39	2.99	3.03	2.34	
C5	2.43	3.18	3.31	3.74	4.05	4.47	4.07	3.72	2.73	2.76	2.19	1.40	
C6	2.42	3.01	3.61	4.01	4.08	4.27	3.94	3.64	3.14	2.41	1.79	0.95	
C7	2.23	2.77	3.07	3.51	3.75	3.72	3.51	3.68	2.49	1.78	1.27	0.81	
C8	2.18	2.97	3.51	3.85	3.49	3.92	3.75	2.48	2.16	1.45	1.20	0.64	
C9	2.71	2.67	3.54	3.93	4.09	3.77	3.00	1.98	1.70	0.63	1.09	1.16	
C10	2.64	3.18	3.61	4.12	4.06	3.60	2.48	2.39	0.94	1.19	1.10	1.22	

先记住这张表：这一讲的每一个部件，都要拿它来试一遍

14 / 96

PART 02

R 的三套图形系统

同一个 R，为什么画出来的图差别这么大

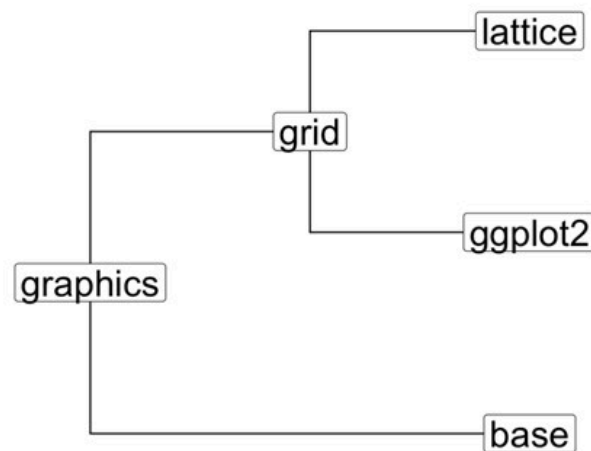
- base: 纸和铅笔，没有橡皮
- grid: 可以拆开重组的图形对象
- lattice: 多元数据的 Trellis 图
- ggplot2: 图形语法

15 / 96

R 的图形能力是分层的

底层负责画，高层负责想

- 底层系统：base、grid —— 提供画笔与画布
- 高层系统：lattice、ggplot2 —— 提供语法与思路
- 高层系统都建立在 grid 之上
- 理解分层，才知道出问题该去改哪一层



R 图形系统的层次关系

LOW LEVEL

底层：base 与 grid

两者都提供画笔，但对「已经画上去的东西」态度不同

base

- R Core Team 维护
- 最经典的绘图函数：plot / hist / boxplot
- 像纸和铅笔，没有橡皮
- 画上去就固定了，改不了
- 很多老包和老教材还在用

grid

- Paul Murrell 开发
- 产出可编辑的图形对象 (grob)
- 可以事后修改、删除、重排
- lattice 与 ggplot2 的基础
- 现代 R 图形的实际底座

一句话：base 画完就完了，grid 画完还能改

17 / 96

lattice: 为多元数据而生

三行代码，一张按大洲分面的 Trellis 图

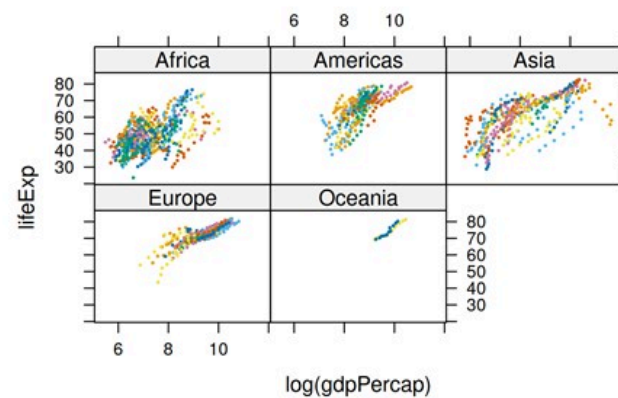
```
gapminder <- read.csv(  
  "https://ramnathv.github.io/pycon2014-r/visualize/data/gapminder.csv")  
library(lattice)  
xyplot(lifeExp ~ log(gdpPercap) | continent,  
  group = country, data = gapminder,  
  cex = 0.3, pch = 20, layout = c(3, 2), as.table = TRUE)
```

- Trellis 图形来自 S-PLUS 的统计图形传统

- 按分组变量切成面板，每个面板重复同一张图

- 擅长多变量、条件分布与分组间的横向比较

- 作者：Deepayan Sarkar
lattice 包的维护者



代码实跑结果：按 continent 分面、按 country 着色

lattice: 一个 panel 函数，四层图形

公式决定「谁对谁作图、按什么分组」，panel 函数决定每个面板里画什么

```
panel.fun<- function(x, y, groups, subscripts, ...) {
  panel.stripplot(x, y, ..., groups= groups,
    subscripts= subscripts)
  panel.lmline(x, y, col = "steelblue", lwd= 2)
  panel.spline(x, y, col = "black", lwd= 3)
  panel.rug(x, y, col = "firebrick")
}

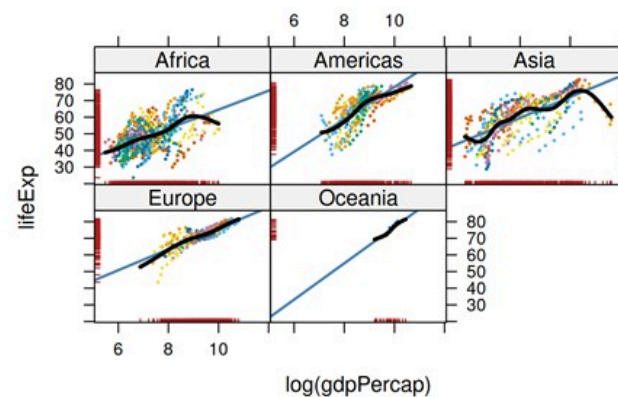
xyplot(lifeExp ~ log(gdpPercap) | continent,
  group = country, data = gapminder,
  cex = 0.3, pch = 20, panel = panel.fun,
  layout = c(3, 2), as.table = TRUE)
```

• $y \sim x$: y 对 x 作图

• $y \sim x | g$: 按 g 分面

• panel = 自定义面板函数

• 一次调用出整套面板图



代码实跑结果：每个面板叠加回归线、样条曲线与边缘 rug

• 缺点：语法自成一套，与数据操作衔接不自然

ggplot2: 把作图变成一门语言

作者 Hadley Wickham, 2005 年提出图形语法

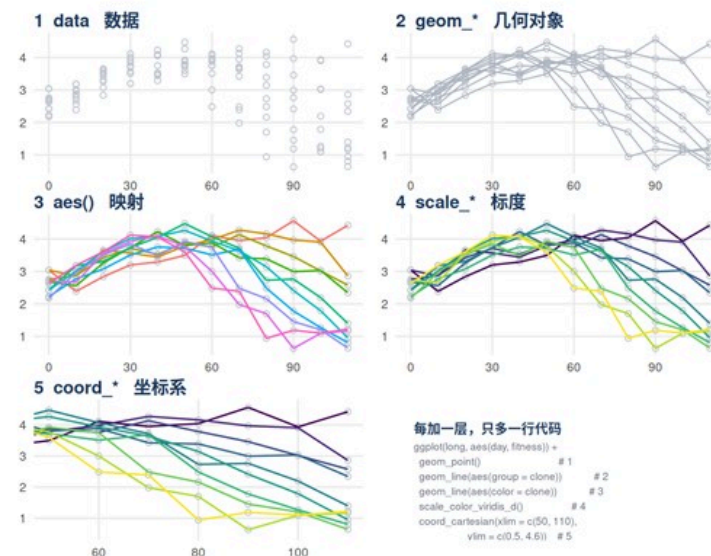
- 遵循 Wilkinson (2005) 的图形语法

- 用高层抽象描述一张图, 而不是逐个命令堆叠

- 自带 theme 系统, 统一调整外观

- 扩展包生态极其活跃, 主题与图层可插拔

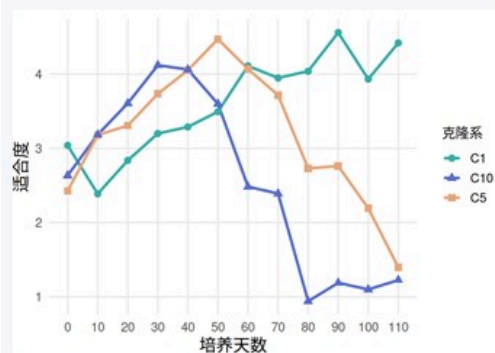
- 生物信息学期刊配图的主流工具



还是第 14 页那 120 个数: data → geom → aes → scale → coord

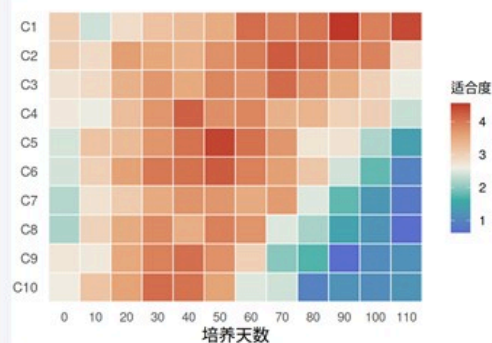
ggplot2 的扩展生态

官方收录的扩展包就有上百个，覆盖几乎所有图形类型



适合度轨迹 (geom_line)

取其中 C1/C5/C10 三条曲线，用 color 区分



适应性热图 (geom_tile)

同一批数据的 10 个克隆系 × 12 个时间点

扩展包 = 同一套语法的插件

```
install.packages("ggplot2")
```

```
install.packages("ggtree")
```

```
install.packages("patchwork")
```

```
install.packages("aplot")
```

```
install.packages("ggrepel")
```

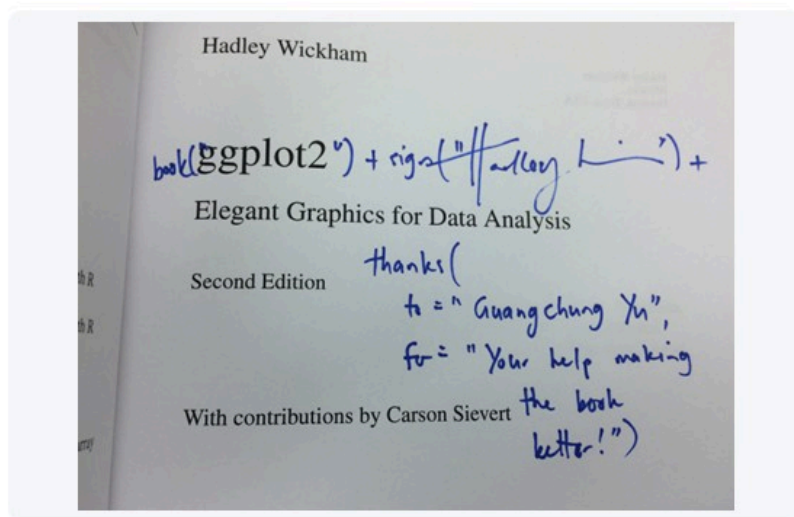
装完即插即用，语法不变

生态即插件，装完即用

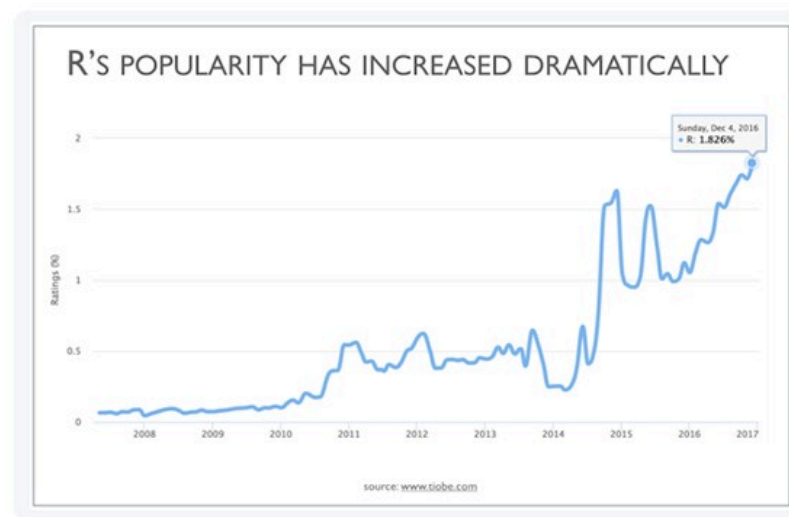
换包不换语法，图层照常叠加

扩展包解决的是「语法之外」的需求

只要语法没变，新图形就能无缝拼进来



ggplot2 扩展包画廊 (ggplot2-exts.org)



R 在数据科学中的定位

ggplot2 4.0: 底层重写, 语法不变

2025 年 9 月发布, 是十年来最大的一次内部改造 —— 你要学的东西没有变

变化	以前	现在	对你的影响
对象系统	基于 S3	迁移到 S7	包作者需要适配, 用户基本无感
扩展机制	ggplot_add()	update_ggplot()	写扩展包的人要改代码
主题模型	theme 元素各自为政	ink / paper 模型	主题继承更一致, 配色更省事
几何参数	各 geom 属性各自为政	element_geom() 统一	一处设置, 全局生效
分面布局	facet_wrap 4 种	8 种 + space	分面排列更灵活
坐标反向	需改 scale	coord_* 增加 reverse	写起来更直观
箱线/小提琴	参数较少	新增样式参数	细节控制更细
label.size	沿用多年	改名 linewidth	老代码会有提示

升级前先看 NEWS; 扩展包作者要留适配时间, 用图的人照旧写代码

COMPARE

三套系统怎么选

新手只需要记住最后一列

系统	抽象层次	擅长	学习成本	建议
base	低	快速画个草图、老代码维护	低	了解即可
lattice	高	多变量分面、条件分布	中	知道它存在
ggplot2	高	绝大多数出版级图形	中	重点掌握

把精力集中在 ggplot2 上，是最划算的选择

24 / 96

SUMMARY

第二部分小结

三个层次，一条主线

底层 base / grid

负责把像素画到画布上，grid 让图形对象可以被修改

中层 lattice

用公式与分面解决多元数据对比，是 ggplot2 的前身思想

高层 ggplot2

用语法描述图形，把「怎么画」变成「画什么」；2025 年的 4.0 重写了底层，语法没变

共同趋势

抽象层次越高，越容易复用、批量与自动化

抽象不是为了变复杂，是让你少写重复的代码

25 / 96

PART 03

图形语法

一张图，其实只有五个部件

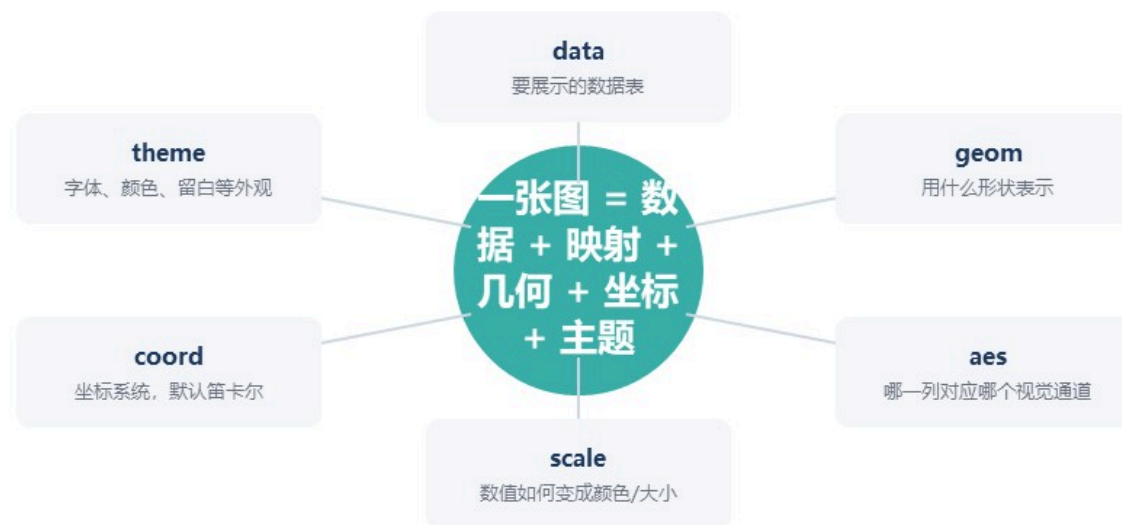
- 数据: data
- 几何对象: geom
- 映射: aesthetic & scale
- 坐标系: coord
- 主题: theme

26 / 96

THE GRAMMAR OF GRAPHICS

图形语法的核心主张

图形不是「画」出来的，是「描述」出来的



把这五个部件说清楚，任何一张图你都能拆开、也能改出来

27 / 96

BASIC ELEMENTS

五个部件在一张图里的位置

对照这张图，找一找每个部件在哪里

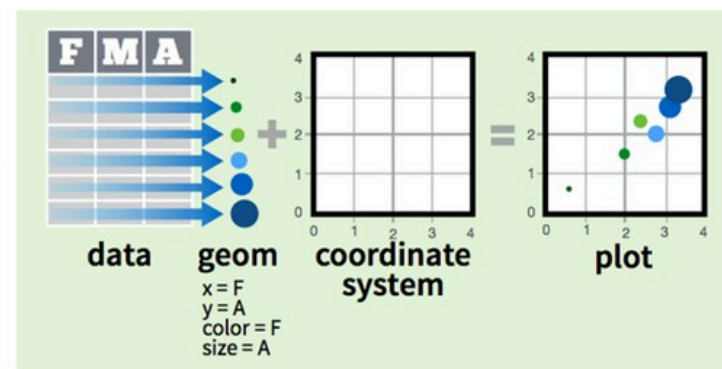
- data: 最底层的表格，一行一个观测

- aes: 把列名映射到 x、y、color、size

- coord: 大多数 geom 用默认笛卡尔坐标

- geom: 图中你能看到的点、线、柱

- scale: 决定数值怎么变成具体颜色或大小



ggplot2 的基本元素

数据长宽格式与元素的关系

tidyr 的 `pivot_longer` / `pivot_wider` 决定了映射的难易

- 宽格式：每个变量一列，适合录入与阅读

- `pivot_longer()`：宽变长，把多列压成两列

- `ggplot2` 更喜欢长格式，因为它天然对应映射

- 长格式：一行变量名 + 一列数值，适合映射

- `pivot_wider()`：长变宽，把一行拆成多列

```
iris2 |> tidyr::pivot_longer(cols=c(Species, sample), names_to='AttrType') -> iris3
iris3
# A tibble: 600 x 4
#   Species sample AttrType value
#   <fct>   <chr>   <chr>   <dbl>
1 setosa setosa1 Sepal.Length 5.1
2 setosa setosa1 Sepal.Width 3.5
3 setosa setosa1 Petal.Length 1.4
4 setosa setosa1 Petal.Width 0.2
5 setosa setosa2 Sepal.Length 4.9
6 setosa setosa2 Sepal.Width 3
7 setosa setosa2 Petal.Length 1.4
8 setosa setosa2 Petal.Width 0.2
9 setosa setosa3 Sepal.Length 4.7
10 setosa setosa3 Sepal.Width 3.2
# ... 590 more rows
# Use 'print(n = ...)' to see more rows
iris3 |> tidyr::pivot_wider(id_cols=c(Species, sample), names_from=AttrType, values_from=value)
# A tibble: 150 x 6
#   Species sample Sepal.Length Sepal.Width Petal.Length Petal.Width
#   <fct>   <chr>   <dbl>   <dbl>   <dbl>   <dbl>
1 setosa setosa1 5.1 3.5 1.4 0.2
2 setosa setosa2 4.9 3 1.4 0.2
3 setosa setosa3 4.7 3.2 1.3 0.2
4 setosa setosa4 4.6 3.1 1.5 0.2
5 setosa setosa5 5 3.6 1.4 0.2
6 setosa setosa6 5.4 3.9 1.7 0.4
7 setosa setosa7 4.6 3.4 1.4 0.3
8 setosa setosa8 5 3.4 1.5 0.2
9 setosa setosa9 4.4 2.9 1.4 0.2
10 setosa setosa10 4.9 3.1 1.5 0.1
# ... 140 more rows
# Use 'print(n = ...)' to see more rows
```

`pivot_longer()` 把宽表压长，`pivot_wider()` 把长表摊宽

数据：一切从这里开始

ggplot2 要求数据是 data.frame 或 tibble

- 一行 = 一个观测 (observation)

- 一列 = 一个变量 (variable)

- 同一列只能是一种类型，否则会被强制转换

- 数据没整理好，图一定画不顺

- 先想清楚「谁是 x，谁是 y，谁分组」再动手

Tree species	Site	DBH (cm)
<i>Acer rubrum</i>	A	15
<i>Acer rubrum</i>	B	8
<i>Acer rubrum</i>	C	30
<i>Acer rubrum</i>	D	27
<i>Quercus alba</i>	A	29
<i>Quercus alba</i>	B	17
<i>Quercus alba</i>	C	14
<i>Quercus alba</i>	D	42
<i>Pinus taeda</i>	A	10
<i>Pinus taeda</i>	B	19
<i>Pinus taeda</i>	C	25
<i>Pinus taeda</i>	D	23

整洁数据 (tidy data) 的形状

30 / 96

几何对象：图里的形状

同一个数据集，换个 geom 就是另一张图

① geom_point: 散点

② geom_line: 折线

③ geom_bar: 柱状

④ geom_boxplot: 箱线

⑤ geom_smooth: 拟合曲线

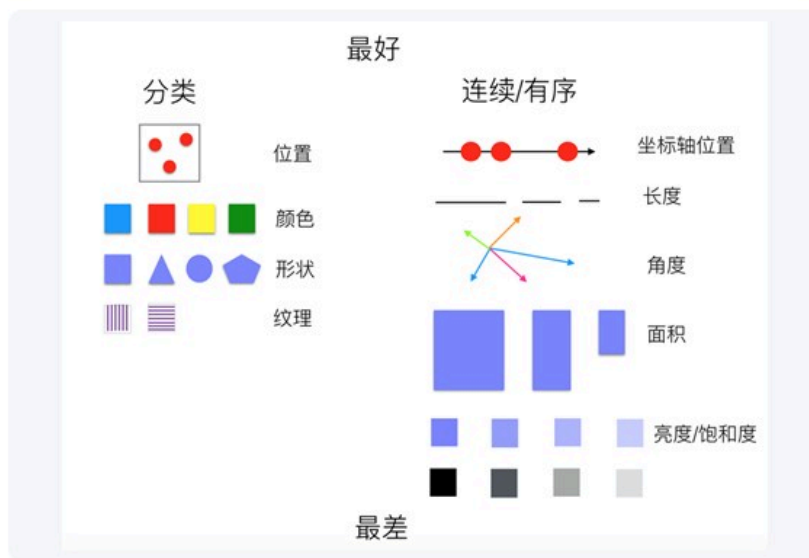
• geom_* 决定「画成什么形状」



同一份数据、同一套映射，只换 geom_* (编号与左栏同序)

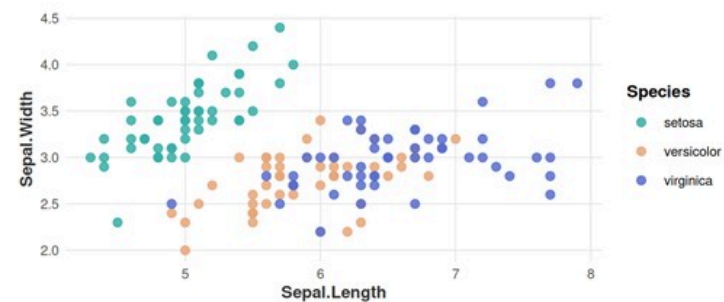
映射：把数据列接到视觉通道上

这是 ggplot2 最关键的一步



分类看形状与颜色，连续看位置与长度

```
# aes() 里写列名 = 把列映射到视觉通道
ggplot(iris, aes(Sepal.Length, Sepal.Width,
  color = Species)) +
  geom_point()
```

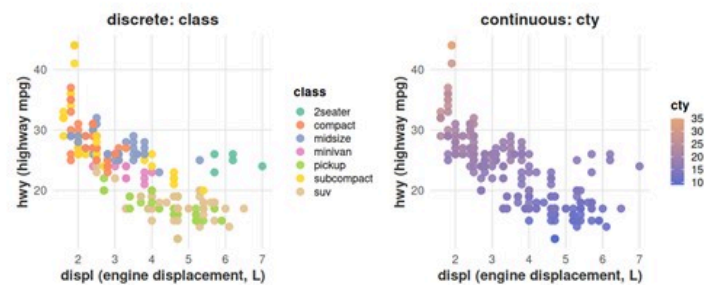


把列名写进 aes(), 数据就自动接到视觉通道上

标度：数值怎么变成视觉

颜色、大小、形状、透明度都由 scale 控制

```
# 离散：类别 → 不同颜色的色块
+ scale_color_brewer(palette = "Set2")
# 连续：数值 → 平滑的色条
+ scale_color_gradient(low = "#5B6FCF")
```

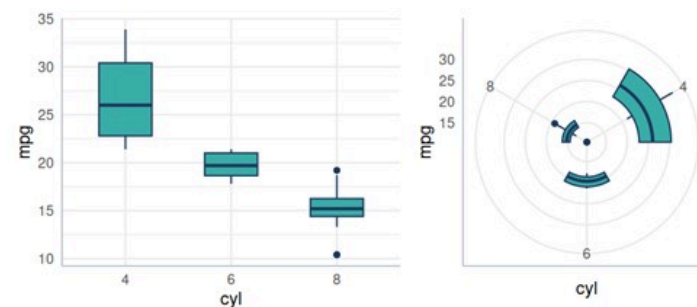


同一份数据：离散标度给色块，连续标度给色条

坐标系：默认笛卡尔

少数情况下需要换一个空间

```
ggplot(mtcars, aes(factor(cyl), mpg)) +  
  geom_boxplot() +  
  coord_cartesian() # 左：直角坐标 |  
  coord_polar()    # 右：极坐标
```



同一数据同一映射，换坐标系就大变样：coord_cartesian() vs coord_polar()

34 / 96

TEMPLATE

一张 ggplot2 图的骨架

背下这个模板，80% 的图都能照着写

- `ggplot(data, aes(...))` 起手

- `+ geom_*()` 叠加图层

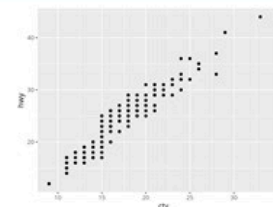
- `+ scale_*()` 调整标度

- `+ coord_*()` 换坐标系

- `+ theme_*()` 收拾外观

- `+ labs()` 写标题与轴标签

```
library(ggplot2)
ggplot(data = mpg, aes(cty, hwy))
  + geom_point()
  + coord_flip()
  + theme_minimal(base_size = 14)
```



代码与它画出的图 (mpg: cty vs hwy 散点)

35 / 96

TEMPLATE

同一个模板，写成代码长这样

注意 + 的位置：必须放在行尾

```
library(ggplot2)

ggplot(df, aes(x = gene, y = expression, color = group)) +
  geom_point(size = 3) +
  scale_color_manual(values = c(ctrl = '#3AAFA9',
                                treat = '#E8A87C')) +
  coord_flip() +
  labs(title = '差异表达基因', x = NULL, y = 'log2(TPM)') +
  theme_minimal(base_size = 14)
```

- ggplot() 只声明数据与全局映射
- 每个 + geom_*() 增加一个图层
- 图层顺序决定遮挡关系，后画的在上面
- scale_*() 决定数值怎么变成颜色/大小
- theme_*() 只改外观，不影响数据

记住三件事：数据在 ggplot()，形状在 geom_，外观在 theme_

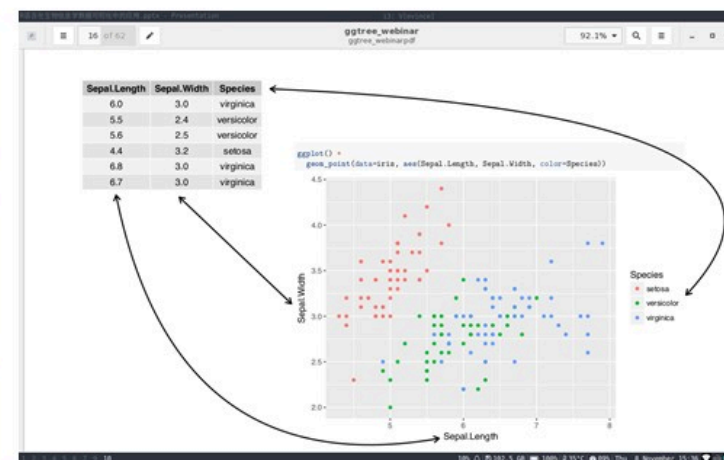
36 / 96

AESTHETIC MAPPING

映射变量：哪一列接到哪里

一张图可以同时映射很多列，但要克制

- x / y: 位置，最基础的映射
- color / fill: 颜色，适合分组变量
- size: 大小，适合连续数值但不宜超过 3 档
- shape: 形状，适合不超过 6 个类别
- alpha: 透明度，处理点重叠很有效
- 映射越多，读者要解码的信息就越多



可用的视觉通道一览

37 / 96

NON-VARIABLE ADJUSTMENT

非变量调整：写死一个值

这些参数要放在 aes() 外面

- `geom_point(color = 'red')`
所有点都是红色

- 写死用常量，映射用 `aes()`
，位置决定含义

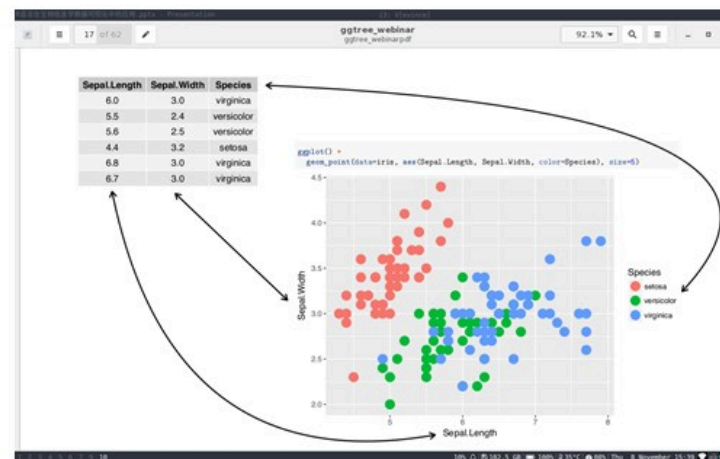
- 判断口诀：这个值跟数据有关吗？无关就写死

- `geom_point(aes(color = 'red'))`
会多出一个红色图例

- 常见写死参数：
`color`、`size`、`alpha`
`linewidth`、`shape`

`ggplot()` +

```
geom_... ( [data = data frame]  
           [aes(variable mappings)]  
           [non-variable adjustments] )
```

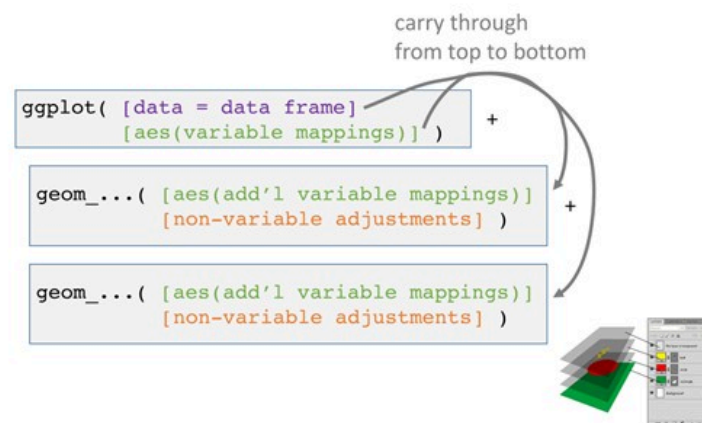


上：模板的第三格（橙色）；下：写死值与映射值的区别

复杂图也是同一个模板

只是图层更多、参数更长

- 图层越多，越要写注释分隔
- 把公共的 `aes()` 放在 `ggplot()` 里
- 只在个别图层变化的映射，写在那个 `geom` 里
- 建议每层一行，行尾留 +
- 复杂图先跑通最简版本，再逐层加



复杂图的组织方式

BUILDING A PLOT

一张图是怎么一层层长出来的

顺序不是随便排的，每一步都对应一个部件

1 起手

`ggplot(data, aes)`

2 加形状

`+ geom_point()`

3 调标度

`+ scale_color_*`

4 换坐标

`+ coord_flip()`

5 收拾外观

`+ theme_minimal()`

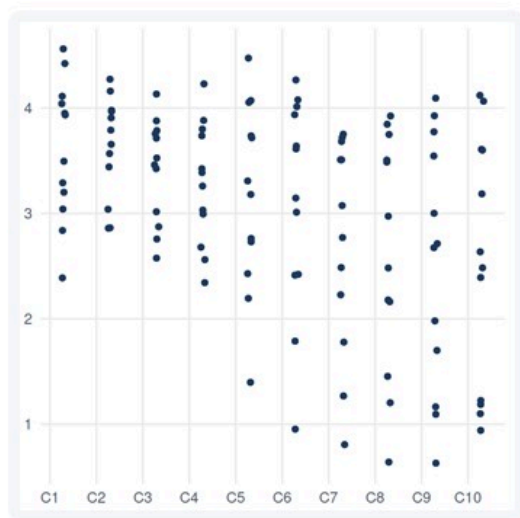
写 ggplot2 就像搭积木：一块一块加，随时可以停下来看效果

40 / 96

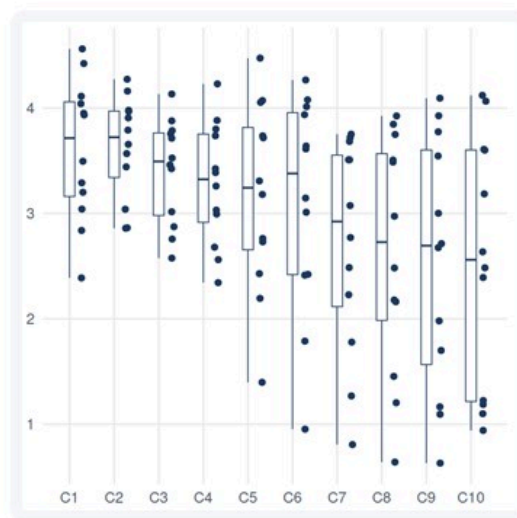
STEP BY STEP

同一张图，叠加过程逐帧看

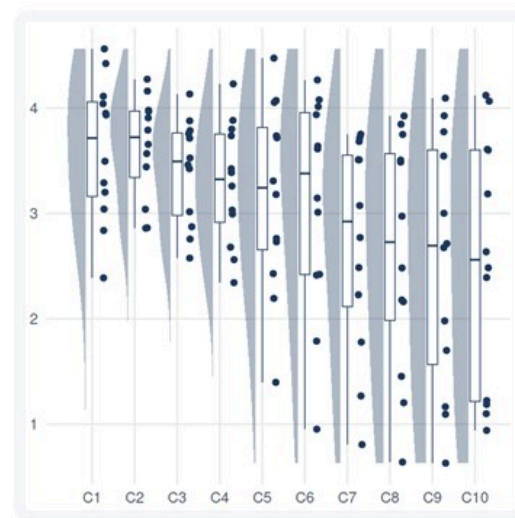
一张云雨图，一层一层叠出来



① `geom_point()`: 先摆一列原始点



② `+ geom_boxplot()`: 加一只箱子



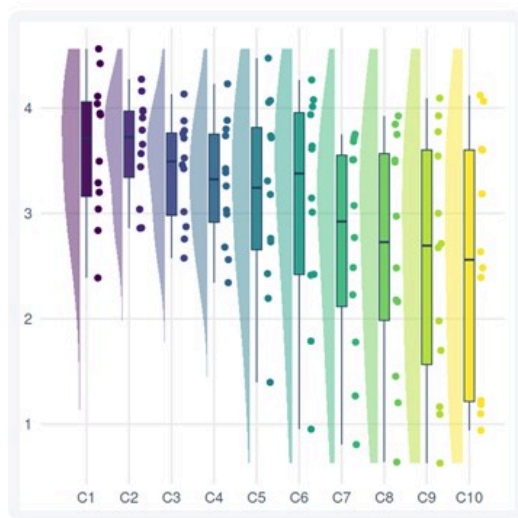
③ `+ density()` 手算多边形: 加半片云

41 / 96

STEP BY STEP

继续叠加，直到成品

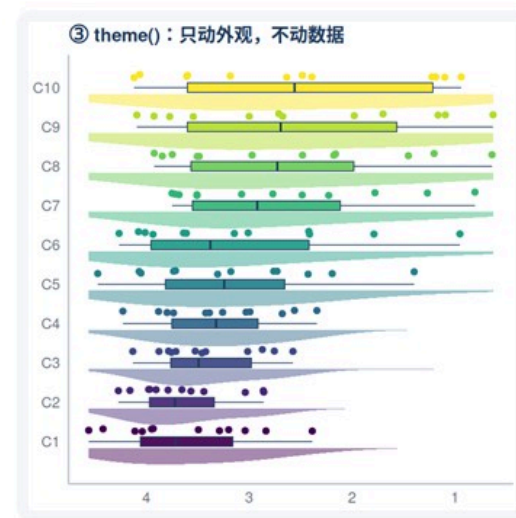
① ~ ③ 立起了形状，④ ~ ⑥ 让它能读



④ 上色: `fill = clone + viridis` 色阶



⑤ `coord_flip()`: 横过来，云飘在点上



⑥ `theme()`: 只动外观，不动数据

42 / 96

SUMMARY

第三部分小结

五个部件，一套模板

data

整洁数据：一行一个观测，一列一个变量

geom

决定形状；geom 之外的一切都只是修饰

aes / scale

映射与标度：数据列如何变成视觉特征

coord / theme

坐标系与外观，影响呈现不影响结论

模板

`ggplot() + geom_*() + scale_*() + coord_*() + theme_*()`

能拆成五个部件，就没有看不懂的 ggplot2 图

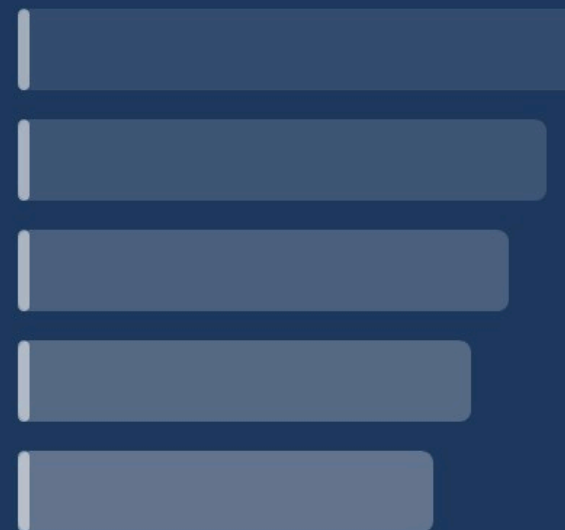
43 / 96

PART 04

常用部件速查

geom / scale / coord / theme 各有哪些常用选项

- geom: 形状的菜单
- scale: 标度的旋钮
- coord: 坐标系的选择
- theme: 外观的开关



44 / 96

TYPES OF GEOMS

geom 的菜单：从点到地图

挑 geom 的本质是挑「用什么形状表达数据」

- 点: `geom_point`、`geom_jitter`

- 面: `geom_bar`、`geom_area`、`geom_polygon`

- 文本与标注: `geom_text`、`geom_label`、`geom_rug`

- 线: `geom_line`、`geom_smooth`、`geom_segment`

- 分布: `geom_boxplot`、`geom_violin`、`geom_density`

- 还有 `geom_map`、`geom_sf` 等空间图形

同一份数据，换 geom 就是换视角

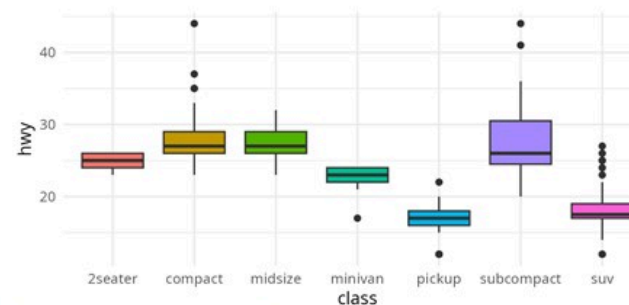
```
ggplot(data = mpg, aes(class, hwy))
```

```
+ geom_boxplot(fill = "#3AAFA9") # 分布
```

```
+ geom_violin() # 密度
```

```
+ geom_jitter() # 个体
```

```
+ geom_bar(stat = "identity") # 比较
```



代码 + 它画出的图: boxplot (48 页换 violin)

45 / 96

HOW TO LOOK UP

忘了参数? 直接问 R

?geom_point 比搜索引擎更快, 而且一定准确

- ?geom_point 或 help(geom_point) 打开帮助页
- 帮助页的 Aesthetics 一节列出所有可用映射
- Examples 一节可以直接复制运行
- 看 aes 后面的 . 是 required 还是 optional
- 养成习惯: 先用 ?, 再搜索

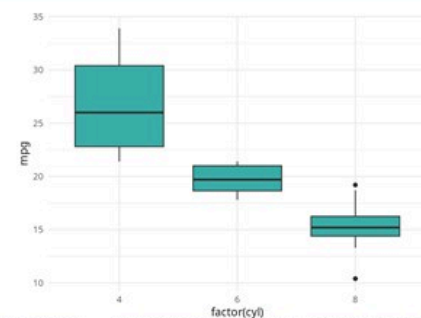
忘了参数? 直接问 R

```
?geom_point# 或 help(geom_point)
```

```
library(ggplot2)
```

```
ggplot(data = mtcars, aes(mpg)) +
```

```
  geom_boxplot(fill = "#3AAFA9", outline.col = "#1F3A5F")
```



?geom_point 的 Examples 一节直接运行 → 上图的代码画出的 boxplot

46 / 96

CHEATSHEET

常用 geom 与适用场景

拿到数据先问：我要展示的是关系、分布还是组成？

想展示	推荐 geom	数据要求	常见坑
两变量关系	geom_point / geom_smooth	两个连续变量	点重叠严重时用 jitter
分组比较	geom_boxplot / geom_violin	一个分类 + 一个连续	样本量小时箱线图会骗人
类别计数	geom_bar / geom_col	一个分类变量	bar 会自己统计，col 需要先汇总
分布形状	geom_histogram / geom_density	一个连续变量	binwidth 会显著改变观感
随时间变化	geom_line	一个时间 + 一个数值	别忘了 group = 分组变量
组成比例	geom_bar(position = 'fill')	两个分类变量	堆叠顺序会影响可读性

选 geom 之前，先用一句话说清「我要让读者看到什么」

47 / 96

SCALE

标度：命名规律是 `scale_通道_类型`

通道是 color / fill / size / x / y, 类型是 discrete / continuous

- `scale_color_manual()`: 手工指定离散颜色

- `scale_fill_gradient()`: 连续填充色阶

- `scale_size_continuous`
(range = c(1, 8))
控制大小范围

- `scale_x_continuous`
(limits = c(0, 100))
限定范围

- `scale_*_discrete`
与 `scale_*_continuous`
不能混用

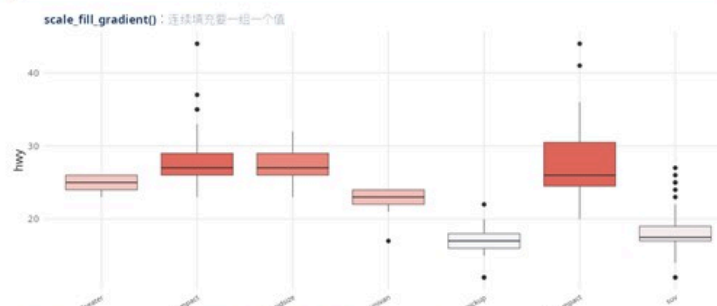
命名规律: scale_通道_类型

```
scale_color_manual(values = c("#3AAFA9"))
```

```
scale_fill_gradient(low = "#F4F6F9", high = "#E0655B")
```

```
scale_size_continuous(range = c(1, 8))
```

```
scale_x_log10()
```



代码 + 它画出的图: fill 要先「一组一个值」, 连续色阶才吃得住

48 / 96

SCALE

换个标度，图的气质就变了

同样的数据，不同的色阶给人的印象完全不同

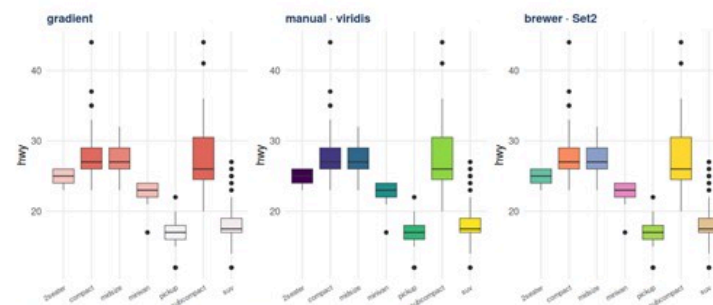
- 连续色阶适合表达强度与梯度
- 离散色板适合表达类别，颜色要能区分开
- 色盲友好配色：viridis、RColorBrewer 的 Set2
- 避免红绿同时出现，约 8% 男性难以区分
- 颜色不是装饰，是编码

同一数据，换色阶，气质就变

```
p + scale_fill_gradient(low = "#F4F6F9", high = "#E0655B")
```

```
p + scale_fill_manual(viridis_pal()(7))
```

```
p + scale_fill_brewer(palette="Set2")
```



代码 + 它画出的图：同一份数据的三种 fill 标度

49 / 96

坐标系：只改呈现，不改数据

coord_flip 是长标签的救命稻草

- coord_cartesian(): 默认直角坐标

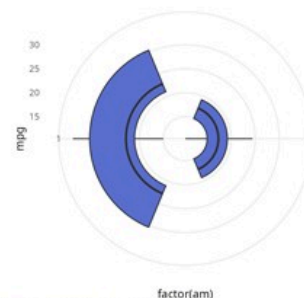
- coord_polar(): 极坐标, 饼图与玫瑰图

- coord_trans(): 坐标轴变换 (如 log)

- coord_flip(): 纵横互换, 基因名很长时必用

- coord_fixed(): 固定纵横比, 画圆、地图、树

```
# 同一数据, 换坐标系, 只改呈现
ggplot(mtcars, aes(factor(am), mpg))
+ geom_boxplot(fill="#5B6FCF")
+ coord_polar() # 直角 → 极坐标
```



代码 + 它画出的图: 上图 of coord_polar 版

50 / 96

什么时候必须换坐标系

当默认坐标让图变形或标签挤不下时

- 基因名很长 → `coord_flip()`
让标签横过来

- 画系统发育树、地图 → `coord_fixed()`

- 这个区别在做局部放大时非常关键

- 要画饼图/玫瑰图 → `coord_polar()`

- 注意: `coord_cartesian`
只缩放视图,
`scale_*_continuous`
会丢数据

```
# 长基因名: 把标签横过来
ggplot(df, aes(value, gene))
+ geom_boxplot()
+ coord_flip() # x/y 互换

# 局部放大: 视图缩放 vs 过滤数据
coord_cartesian(limits=c(0, 10))
# 仍显示完整分布, 只裁视图
```

代码 + 对照: 49 页 polar 图 · 本页 flip / cartesian

51 / 96

THEME

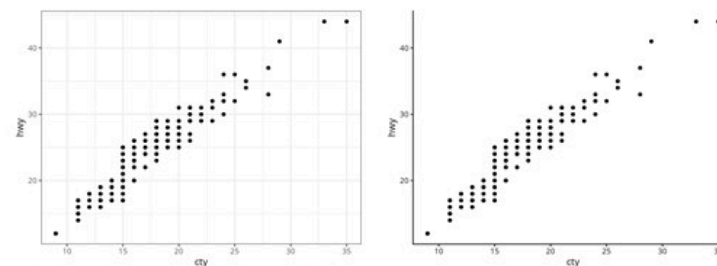
主题：只改外观，不动数据

theme 是让图从「能用」到「好看」的那一步

- `theme_gray()`: 默认，灰底白网格
- `theme_bw()` / `theme_minimal()`: 投稿常用
- `theme_classic()`: 只有坐标轴，没有网格
- `theme(legend.position = 'none')`: 去掉图例
- `element_text` / `element_line` / `element_rect`
控制各类元素

同一数据，换 theme 改外观

```
ggplot(mpg, aes(cty, hwy)) + geom_point() +  
  theme_gray() # 默认  
  theme_bw() # 投稿  
  theme_classic() # 极简  
  theme_minimal() # 无网格
```



左: `theme_bw()` · 右: `theme_classic()`

52 / 96

THEME

主题参数速查

element_* 是 theme 的积木

- element_text(size, face, color, angle)
文字

- element_rect(fill, color):
矩形背景

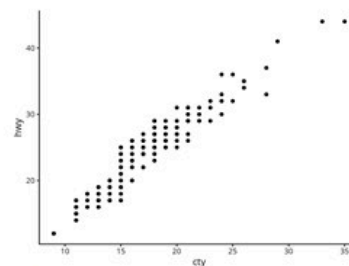
- plot.title /
axis.text.x /
legend.position
是常用槽位

- element_line(color, linewidth): 线条

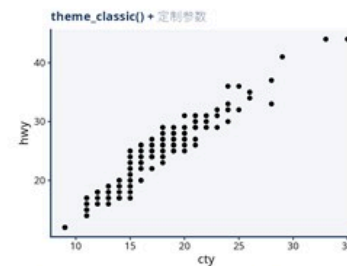
- element_blank(): 直接删掉某个元素

element_* 是 theme 的积木

```
theme(base_size=14)  
+ element_text(size=11, face="bold")  
+ element_line(color="#1F3A5F")  
+ element_rect(fill="#F4F6F9")  
+ element_blank() #删元素
```



theme_classic()



theme_classic() + 定制参数

53 / 96

ink / paper: 主题的新心智模型

ggplot2 4.0 起, 把「画在纸上的墨」和「纸本身」分开

ink

所有可见的前景元素: 点、线、文字、边框

paper

承载这些元素的背景: 画布、面板、网格

from_theme()

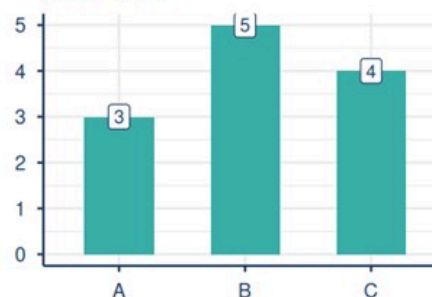
在 `aes()` 映射里直接读当前主题的颜色

element_geom() / margin_auto()

一次性设置某类几何对象的默认参数; 边距也能自动

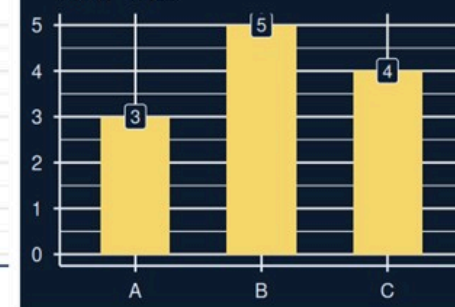
ink / paper 各行其道

浅纸 · 深墨



ink / paper 各行其道

深纸 · 浅墨



只换 paper (背景) 与 ink (文字/线/标签), 数据与映射原封不动

主题从「一堆独立参数」变成「两个可继承的体系」

SUMMARY

第四部分小结

四个部件，各有各的旋钮

geom

决定形状。先用一句话说清要展示什么，再挑 geom。

scale

决定数值到视觉的转换。命名规律 scale_通道_类型。

coord

决定坐标系。长标签、饼图、地图、树都靠它。

theme

决定外观。element_* 是积木；4.0 起改用 ink / paper 两个可继承的体系。

geom 管「画什么」， scale / coord / theme 管「怎么呈现」

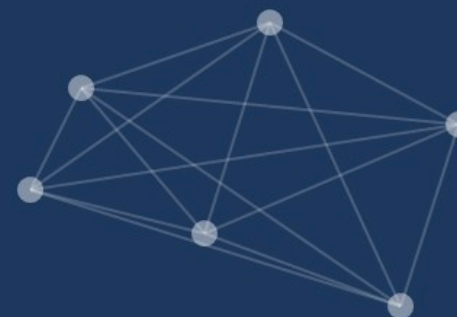
55 / 96

PART 05

复杂图：拼图的本质

不是把图摆在一起，而是让数据互相说话

- 组合图：复杂度来自另一个层次
- 热图 + 树：为什么摆在一起是错的
- 从排版驱动到数据驱动
- aplot: 让拼图自动对齐



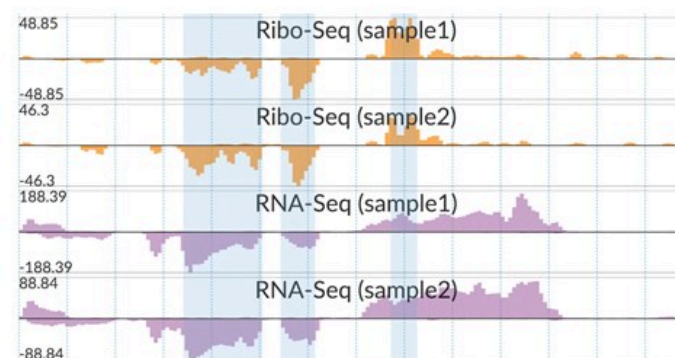
56 / 96

COMPOSITE PLOT

组合图：复杂度在另一个层次

单张图很熟悉，拼在一起就变难了

- 每个子图单独看都没问题
- 难点在于子图之间的对齐、比例与共享轴
- 位置对齐错一点，读者就会误读
- 最麻烦的是：数据变了，对齐关系要不要跟着变
- 这就是「排版驱动」与「数据驱动」的分水岭



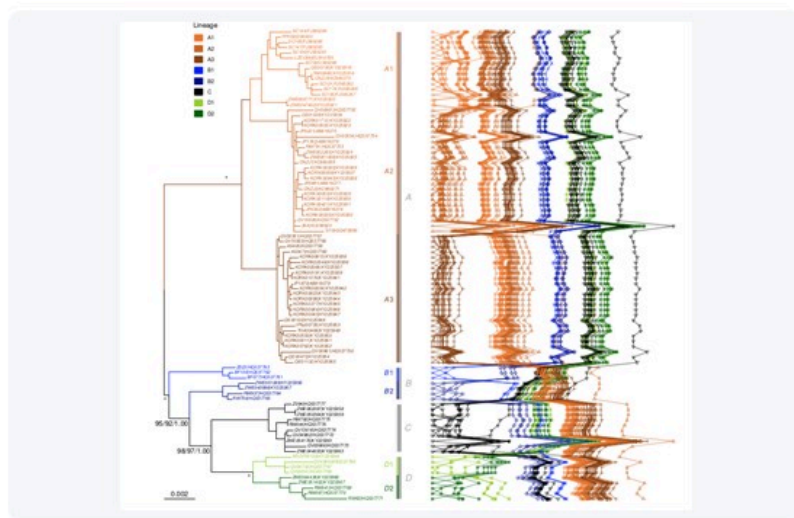
典型的多面板组合图

57 / 96

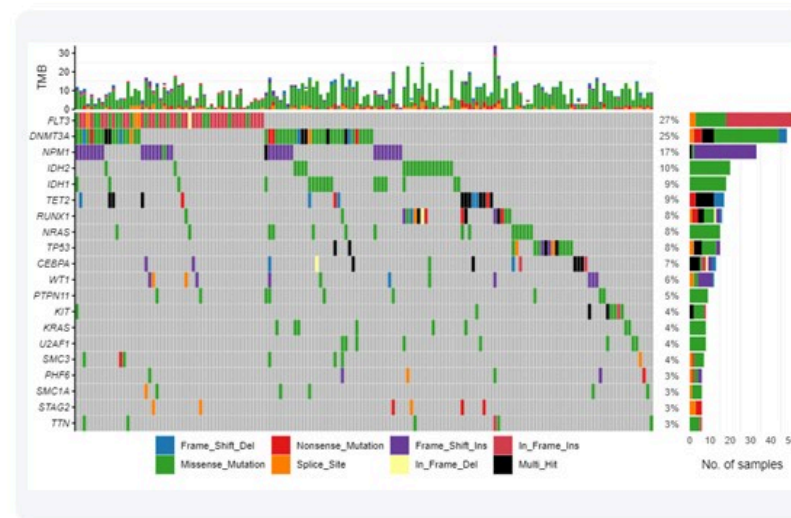
COMPOSITE PLOT

生物信息学里到处都是组合图

越复杂的结论，越依赖多图协同



多组学联合展示

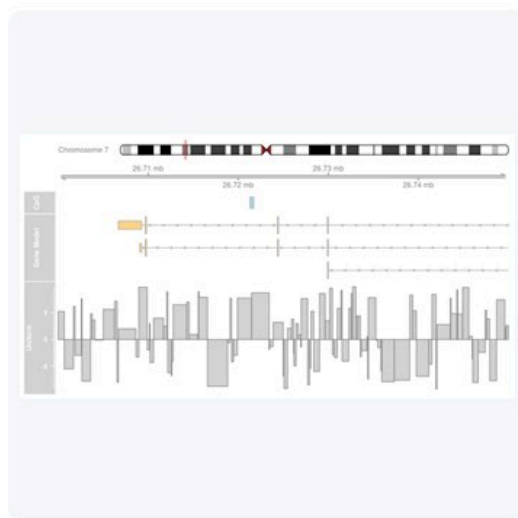


多面板整合结果

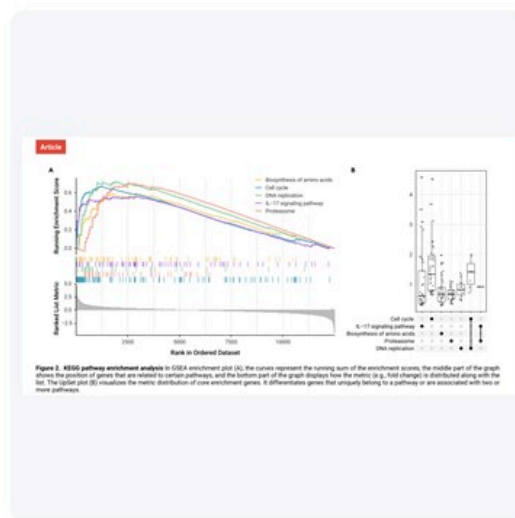
COMPOSITE PLOT

组合方式可以非常多样

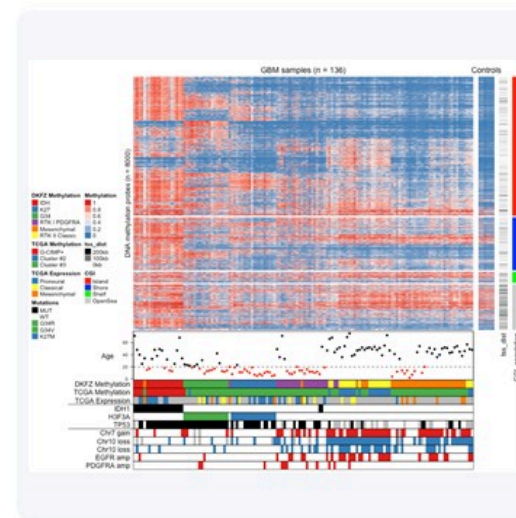
共享轴、嵌套、镶嵌、叠加



共享坐标轴对齐



嵌套式布局



不规则镶嵌

A CLASSIC PROBLEM

热图 + 树：看起来对，其实错了

行顺序不一致，是这类图最常见的错误

- 热图需要按树的结构重新排序行
- 自己动手对齐几乎做不到
- 核心矛盾：图与树是两套独立的坐标系
- 大多数读者并不熟悉树结构
- 只能依赖现成的包，但包又不一定合心意

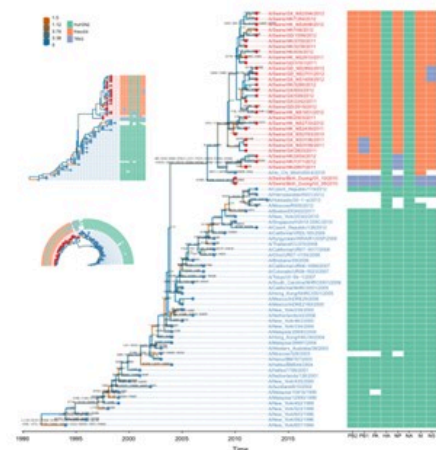


Fig. 5. Time-scaled phylogenetic tree annotated with a matrix of values associated with each locus, in this case the genotypes of H3N2 influenza viruses. The x-axis is the timescale (in units of year) inferred by us. The tree branches are colored by their d_{ij} values (as in the left scale at the top), and the internal nodes (which show the posterior clade probabilities). Tip labels (names) and circles are colored by species (human in blue and avian in red). The genotype, which is shown as an array of colored boxes on the right, is composed of the lineage (either H3N2 or H1N1), colored as in the right legend at the top of the right genome segments of the virus. Any missing segment segments are shown as empty boxes. Small rectangular and fan leaves of the unlabelled tree are shown in the upper and lower insets on the left. Complete codes are available in the Appendix S2.

热图与聚类树的对齐问题

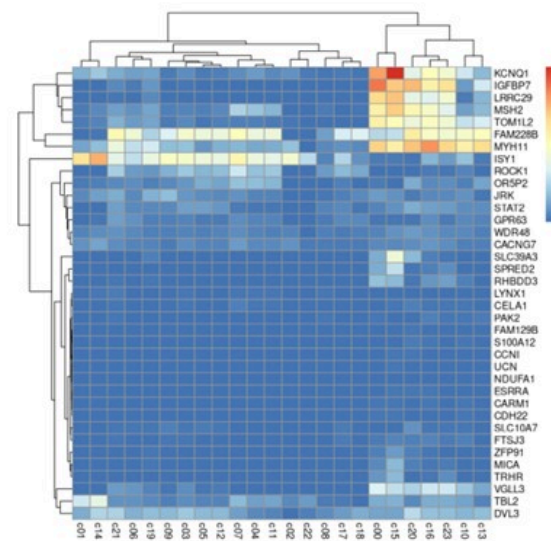
60 / 96

A CLASSIC PROBLEM

如果想自己画呢?

问题就从「怎么画」变成了「怎么对齐」

- ggplot2 能画无排序的热图
- ggtree 能画漂亮的树
- 直接把两张图摆在一起，行顺序对不上
- 手工数行号对齐，数据一更新就全废
- 需要一个机制：拼图时自动同步行顺序



两张图拼在一起后的错位

A STEP FORWARD

退一步想：忘掉「树」

把它当成一个坐标对齐问题

能做到的事

ggplot2 画无序热图，ggtree 画树，两者都很简单

直接拼的后果

行顺序不匹配，图是错的，而且错得很隐蔽

关键提问

能不能让「拼起来」这个动作，自动触发重排热图？

思路转变

从「把两张图放一起」变成「让两张图共享同一套坐标」

这需要什么

一个能理解数据关联的拼图工具

把图形问题抽象成数据问题，往往就找到了出路

62 / 96

APLOT

aplot: 化繁为简，图所未见

自动对齐，是因为它知道两边的数据关系

- aplot 用「数据关联」而不是「位置」来拼图
- 热图与树共享同一套行标识
- 树的行顺序变化会自动传递给热图
- 同一套机制可以复用到任意两张图之间
- 思想来源：ggtree 中系统发育数据整合的经验



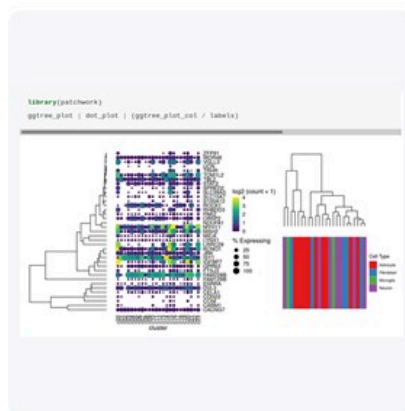
aplot 的对齐机制

63 / 96

APLOT

用 aplot 做的带注释热图

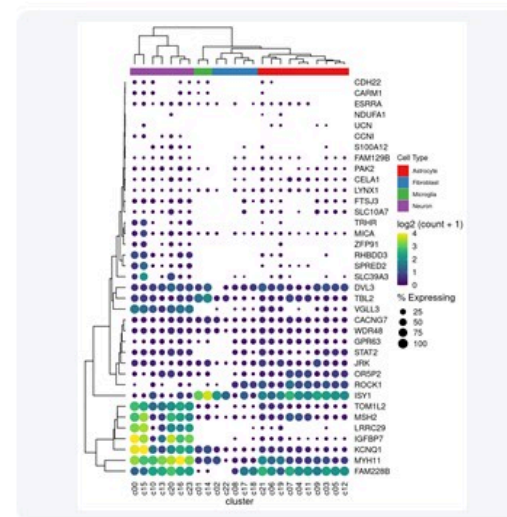
注释信息本身就是数据，不是贴上去的标签



一张普通热图：先画好，
再等注释贴上

```
p_heat <- ggheatmap(mat) # 左图  
library(aplot)  
# 数据关联，自动对齐行/列  
  
p_heat %>%  
  insert_top(ann_track, height = .1) %>%  
  insert_left(tree, width = .2) # 树  
# 成图在右
```

关键：insert_top / insert_left / insert_right



结果：树 + 多轨注释自动对齐

64 / 96

CASE STUDY

给突变全景图加上通路注释

从「谁突变了」到「突变落在哪条通路上」

- oncoplot 展示每个样本的突变分布
- 把基因按通路分组，颜色编码通路信息
- 读者一眼就能看出哪条通路被反复打中
- 这类注释全靠数据关联实现
- 结论的可读性因此提升一个量级

```
library(aplot)
library(aplotExtra)
laml.maf <- system.file("extdata",
  "tcga_laml.maf.gz", package = "maftools")
laml.clin <- system.file("extdata",
  "tcga_laml_annot.tsv", package = "maftools")
maf <- maftools::read.maf(laml.maf,
  clinicalData = laml.clin)
pg <- aplotExtra::oncoplot(maf, 20)
genes <- aplotExtra::get_oncoplot_genes(maf, 20)
X <- clusterProfiler::enrichPathway(genes, 'reactome')
g <- enrichplot::heatplot(x, showCategory=5) + coord_flip()
pg |> insert_right(g, width=3)
```

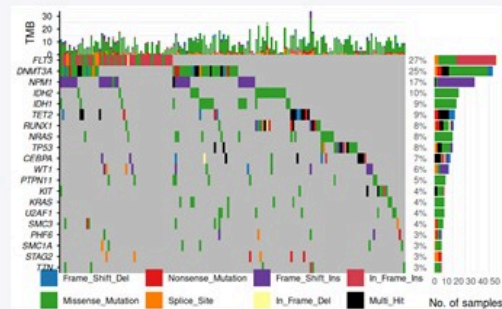
关键代码：aplotExtra::oncoplot 画出图一（pg）
heatplot 出注释轨（g），最后一行 insert_right 把 pg 和 g 拼成图三

65 / 96

CASE STUDY

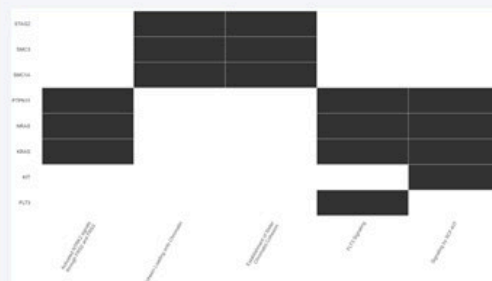
注释层可以无限扩展

只要数据能对上，就能拼进来



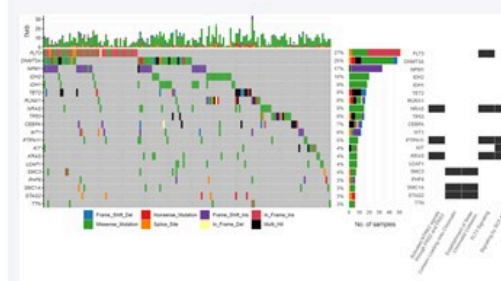
oncoplot: 突变全景图

先有：每个样本×基因



pathway: 通路注释轨

再加一轨：按通路分组着色



整合视图: oncoPrint + 通路注释, 用 insert_right 拼起来

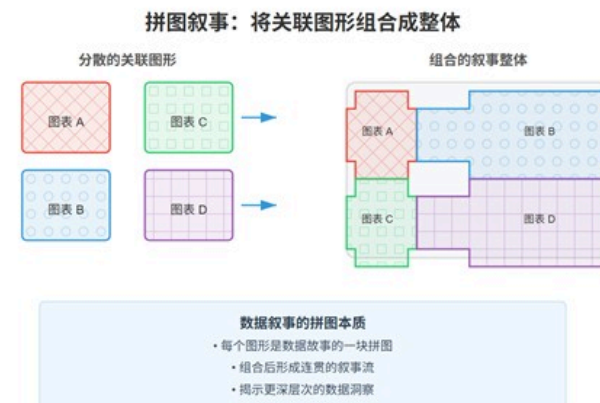
拼起来: 想加多少轨都行

NARRATIVE

复杂数据的多维叙事

本质上还是布局与拼图，核心是数据关联

- 一篇文章的主图往往包含 4—8 个子图
- 子图之间共享数据、共享坐标、共享配色
- 读者沿着一张主图走完整个逻辑链
- 图与图之间的关系，本身就是一种信息
- 把这种关系写进代码，图才能自动更新



多维叙事的主图设计

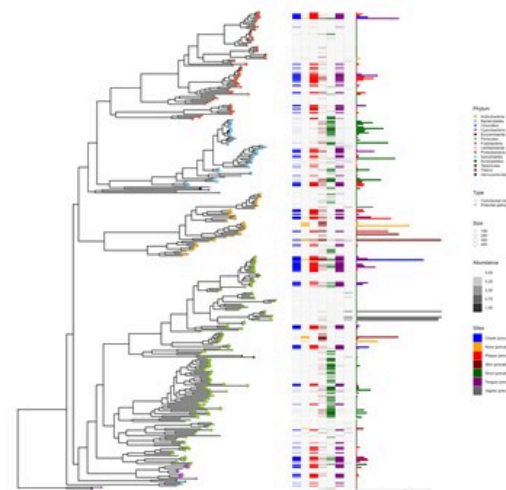
67 / 96

THE SOUL

数据关联：复杂图的灵魂

表象是图，本质是数

- 拼图的两条路：
按位置拼 vs 按数据拼
- 按位置拼：图变了，位置不会跟着变
- 按数据拼：数据变了，所有图一起更新
- 数据关联让图与图之间保持逻辑一致
- 这就是现代可视化工具的核心差异



数据关联示意

68 / 96

OLD vs NEW

旧时代：排版驱动

解决的是「图放在哪里」的问题

排版驱动（布局）

- 关心图在画布上的绝对或相对位置
- 解决「这个图放哪，那个图放哪」
- 只能组合已知信息
- 数据更新后需要重新排版
- 代表：gridExtra、patchwork 的简单用法

数据驱动（关联）

- 关心图与图之间的数据对应关系
- 解决「它们如何在一起」
- 组合过程可以产生新的模式与洞察
- 数据更新后图形自动重排
- 代表：aplot、ggtree 的关联机制

从「在哪里」到「如何在一起」，这是思维方式的转变

69 / 96

NEW ERA

新时代：数据驱动

数据关联带来新的模式和洞察

位置不再重要

你只描述数据怎么对应，位置交给工具算

对齐自动完成

行顺序、坐标范围、分组结构全部同步

组合产生新信息

注释层与主图交叉，能看出单独看不到的规律

可复用

同一套关联逻辑可以迁移到完全不同的图形

可维护

数据换一批，图自动重画，不用重排

从静态到交互

htmlwidgets / plotly / ggiraph 让读者能筛选

从单图到叙事

一张主图承载完整论证链

从手工到声明式

你描述数据关系，工具负责布局

趋势只有一个方向：让人少做机械劳动，多想问题本身

70 / 96

APLOT

aplot: 解放生产力，赋能创造力

把时间从对齐像素里省出来，花在思考数据上

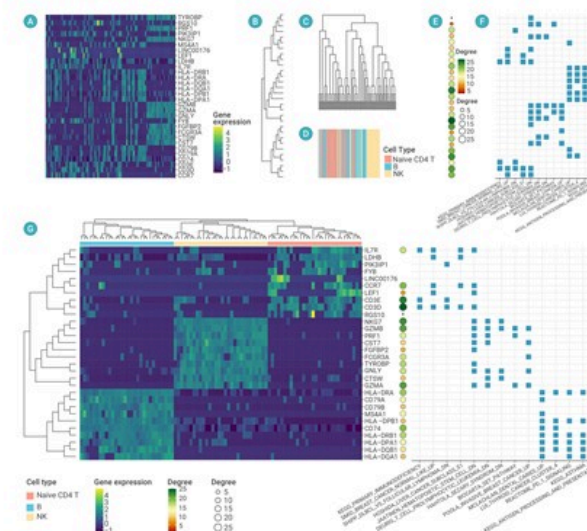
- 省掉手工对齐的时间

- 降低复杂图的维护成本

- 让「试一个新组合」变得便宜

- 创造力来自低成本试错

- 这也是所有好工具的共同目标



aplot 的实际效果

71 / 96

AN IDEA

灵光一现：绘图世界的双城记

一是图层的叠加，像 ggplot2；一是图的关联，像 aplot

- 一类图关心所在的坐标系与位置：地图、树、网络
- 一类图只关心数据的映射：散点、柱状、箱线
- 把两座城对齐，就是连接它们的桥——也是这一页要给的洞见

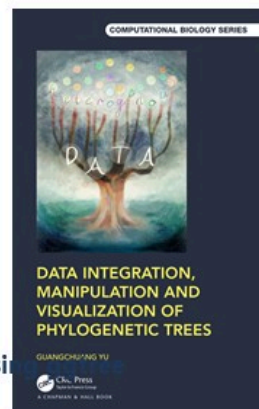
Two methods for mapping and visualizing associated data on phylogeny using

G Yu*, TTY Lam, H Zhu, Y Guan*

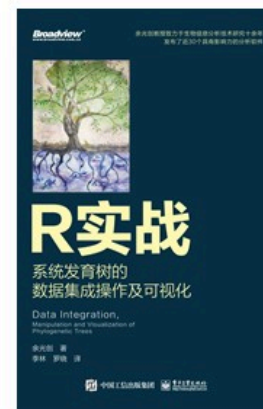
Molecular Biology and Evolution. 2018, 35(12):3041-3043

两种方法：图层的叠加（像 ggplot2）+ 图的关联（像 aplot）

从此，树与它携带的数据可以各画各的，再精准对到一起



英文原版 · CRC Press, 2022



中文译本 · 电子工业出版社, 2023

图的叠加（像 ggplot2）+ 图的关联（像 aplot）= 双城记

72 / 96

EVOLUTION

从 ggtree 的专注到 aplot 的通用

把系统发育数据整合的经验，迁移到通用场景



73 / 96

SUMMARY

第五部分小结

复杂图的难度不在画，而在对齐

组合图的痛点

子图之间的对齐、比例与共享轴，手工做几乎不可维护

排版驱动

只解决「在哪里」，数据更新后要重排

数据驱动

解决「如何在一起」，数据更新后自动重排

aplot 的价值

用数据关联替代位置对齐，把维护成本降到接近零

推广出去

任何两张共享同一标识的图，都可以用同一套逻辑拼起来

先把数据关系想清楚，图自然就对齐了

74 / 96

PART 06

生态与趣味

让学习这件事没那么苦

- 扩展包能解决什么
- ggimage: 把图片放进图里
- emojiFont: 把表情符号画进图里
- 彩蛋: geom_cake

75 / 96

EXTENSIONS

扩展包：官方之外的第二战场

只要语法没变，任何人都能加一种新图形

专业领域图形扩展

`ggtree` 系统发育树与多组学整合
`enrichplot` 富集与通路结果可视化
`ggseqlogo` 序列 motif 图标
`trackViewer` reads 与 peak 轨迹
`EnhancedVolcano` 差异表达火山图

生物信息学有专属的「图形语法插件」

主题与配色扩展

`ggthemes` 期刊与软件风格模板
`ggsci` 期刊推荐配色 (NPG/Lancet)
`viridis` 感知均匀的连续色阶
`showtext` 把任意字体画进图里
`ggtext` 图内 Markdown 富文本

让图从「能用」到「像作品」

通用扩展：复杂度可视化

`patchwork` 多面板拼图 (LAB 04 用)
`aplot` 数据关联拼图 (第 63 页)
`ggimage` 把图片当图形元素 (第 77 页)
`emojifont` 把表情当图形元素 (第 79 页)
`ggrepel` 标签自动避让重叠

遇到需求先搜扩展包，不要自己造

76 / 96

GGIMAGE

ggimage: 把图片放进图里

支持图片文件与图形对象直接作为图形元素

- `geom_image()` 用图片代替点
- 可以把物种照片画在系统发育树上
- 也可以把图标画在柱状图顶端
- 图片既可以是文件，也可以是绘图对象
- 典型用途：让图更直观、更有辨识度

ggimage: Use Image in 'ggplot2'

Supports image files and graphic objects to be visualized in 'ggplot2' graphic system.

Version: 0.3.0
Depends: R (≥ 3.4.0), [ggplot2](#)
Imports: [ggfun](#), [ggplotify](#), [grid](#), [jsonlite](#), [magick](#), [methods](#), [scales](#), [tibble](#), [tools](#), [utils](#)
Suggests: [ape](#), [ggtree](#), [gridGraphics](#), [http](#), [rsvg](#)
Published: 2021-09-29
Author: Guangchuang Yu  [aut, cre]
Maintainer: Guangchuang Yu <guangchuangyu at gmail.com>

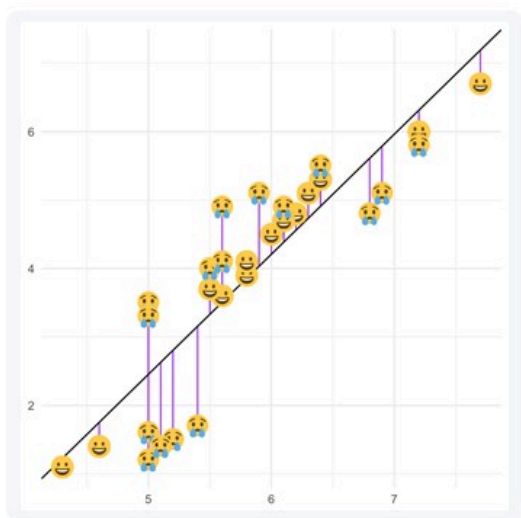
ggimage 的典型用法

77 / 96

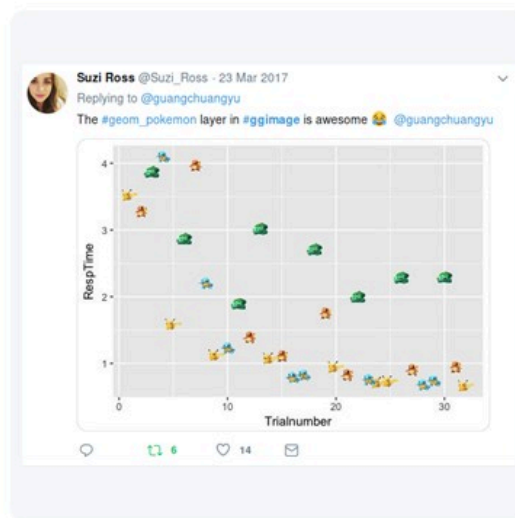
GGIMAGE

图片元素的更多玩法

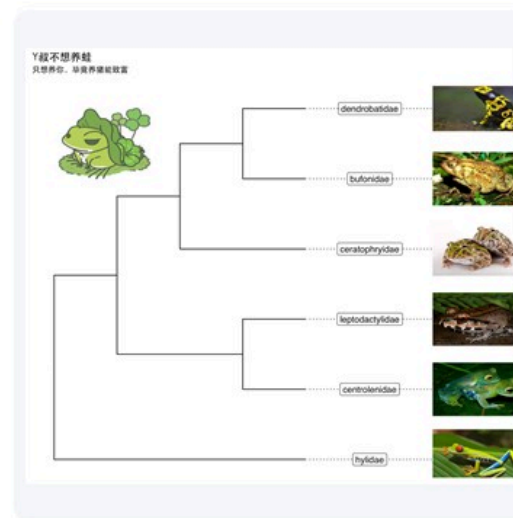
当图里的「点」本身携带信息，图就更有表达力



emoji 笑脸 / 哭脸：表示残差大小



图标化散点图



图片与数据结合

78 / 96

EMOJIFONT

emojifont: 把表情符号画进图里

表情不再是文本，而是和点、线一样的图形元素



geom_emoji(): 左图按坐标打散点，右图用 size 编码数值

```
ggplot() +  
  geom_emoji(alias = "smile",  
             x, y, size)
```

alias: 想用哪个表情

内置丰富的别名: smile / heart / star
/ moon / dog / penguin

x / y / size: 和点、线完全一样的映射

它再次证明: 换一种 geometry, 语法不变

79 / 96

FUN

meme: 学术人的自我修养

严肃科研之余，也需要一点幽默

meme: Create Meme

The word 'Meme' was originated from the book, 'The Selfish Gene', authored by Richard Dawkins (1976). It is a unit of culture that is passed from one generation to another and correlates to the gene, the unit of physical heredity. The internet memes are captioned photos that are intended to be funny, ridiculous. Memes behave like infectious viruses and travel from person to person quickly through social media. The 'meme' package allows users to make custom memes.

Version: 0.2.3
Depends: R (≥ 3.4.0)
Imports: [ggplot2](#), [graphics](#), [grDevices](#), [grid](#), [gridGraphics](#), [magick](#), [methods](#), [showtext](#), [sysfonts](#), [utils](#)
Suggests: [cowplot](#), [ggimage](#), [knitr](#), [rmarkdown](#), [prettydoc](#), [shadowtext](#)
Published: 2021-04-23
Author: Guangchuang Yu  [aut, cre]
Maintainer: Guangchuang Yu <guangchuangyu at gmail.com>

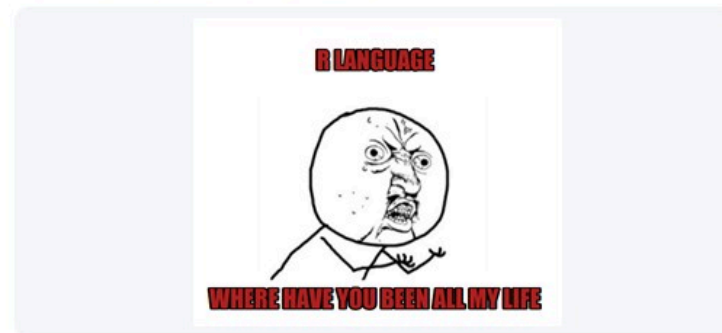
meme 包的帮助页：一张底图 + 上下两行文案

作者 Guangchuang Yu · CRAN v0.2.3

连「玩梗」也能做成一个 R 包

```
f <- paste0("https://y3.ifengimg.com/news_spider/dci_2012",  
           "/06/ad2f18524e4a66377f8aed58f02034db.jpg")  
library(meme)  
meme(f, "R language",  
     "where have you been all my life", color="firebrick")
```

① 三行代码：给底图、给文案、给颜色



② 成品：科研日常，会心一笑

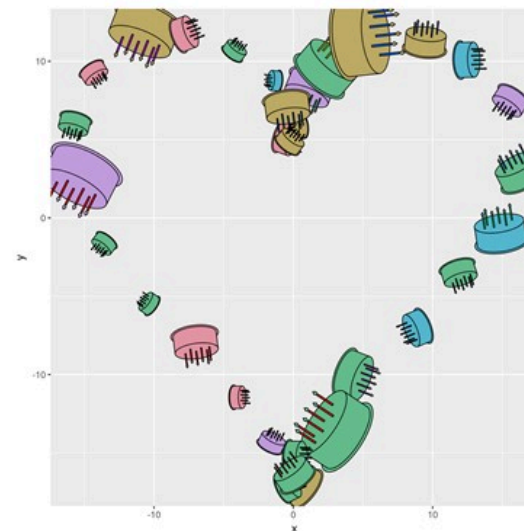
80 / 96

EASTER EGG

彩蛋: geom_cake

不为无益之事，何以遣有涯之生

- 一个「画蛋糕」的彩蛋图层
- 但它把图形语法的思想玩到了极致
- 说明一件事：语法足够通用，就能长出任何东西
- 也是提醒：工具除了有用，也可以好玩
- 兴趣是长期坚持最好的燃料



geom_cake 做出来的蛋糕图·该功能现已并入 ggfun 包

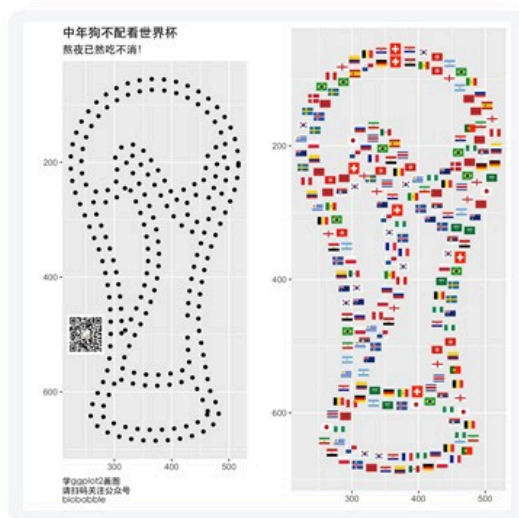
原文: mp.weixin.qq.com/s/lqcqo5YTy1qmXdsulrYQ

81 / 96

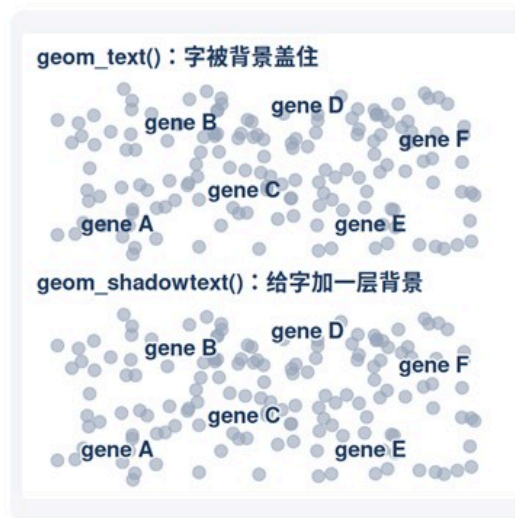
HAVE FUN

Have fun

把学习变成一件愿意重复做的事



ggimage: 用国旗「点」出大力神杯



shadowtext: 给文字加一层背景, 复杂背景下也能突显

82 / 96

SUMMARY

第六部分小结

生态决定了你能走多远

扩展包

语法不变，能力可以无限叠加；遇到需求先搜扩展包

ggimage

图片也是图形元素，能让图更直观

emojifont

把表情符号当作图形元素，画图随取随用

趣味

玩得下去的人，才学得久

态度

把工具当玩具，往往学得最快

先学会语法，再让生态帮你解决剩下的问题

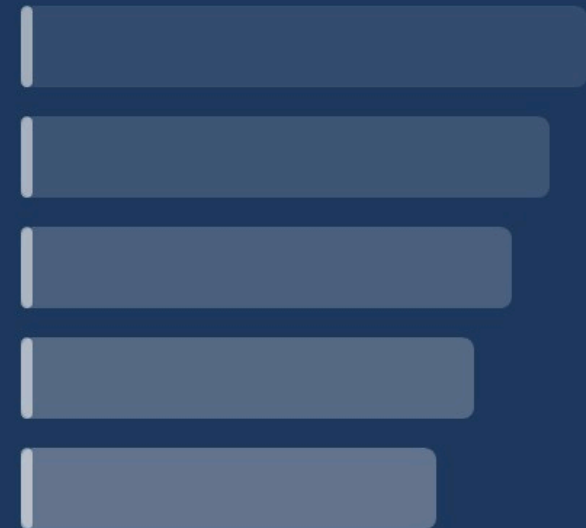
83 / 96

LAB

上机练习

照着做一遍，比看十遍有用

- 练习 1：跑通第一张 ggplot2 图
- 练习 2：换 geom 与 scale
- 练习 3：调整 theme 出一张能投稿的图
- 练习 4：用 patchwork / aplot 拼图



84 / 96

LAB 01

练习 1：跑通第一张图

目标：确认环境正常，理解五个部件在哪里

```
library(ggplot2)

p <- ggplot(mtcars, aes(x = wt, y = mpg, color = factor(cyl))) +
  geom_point(size = 3) +
  labs(title = '车重与油耗', x = '重量', y = '每加仑英里数',
        color = '气缸数')

p

# 思考：data / geom / aes / scale / coord 分别在哪里？
```

第一张图不用好看，先跑通、再理解、最后美化

- 先确认 `library(ggplot2)` 不报错
- 把 `factor(cyl)` 换成 `cyl`，看颜色变成什么样
- 把 `geom_point` 换成 `geom_smooth`，观察区别
- 用 `?geom_point` 查看所有可用映射

85 / 96

练习 2: 换 geom 与 scale

同一份数据, 试着画出四种不同的图

```
# 1. 箱线图: 分组比较
ggplot(mtcars, aes(factor(cyl), mpg)) + geom_boxplot()

# 2. 柱状图: 类别计数
ggplot(mtcars, aes(factor(cyl))) + geom_bar()

# 3. 密度图: 分布形状
ggplot(mtcars, aes(mpg, fill = factor(cyl))) +
  geom_density(alpha=0.5)

# 4. 自定义配色
ggplot(mtcars, aes(wt, mpg, color = factor(cyl))) +
  geom_point() + scale_color_brewer(palette = 'Set2')
```

换个 geom 就是换个视角看同一份数据

- 注意 geom_bar 会自动统计计数, 不需要先汇总
- alpha 控制透明度, 处理重叠很有用
- scale_color_brewer 提供色盲友好色板
- 对比四种图分别适合回答什么问题

练习 3：做一张能投稿的图

目标：把默认的灰底图改成期刊风格

```
ggplot(mtcars, aes(wt, mpg, color = factor(cyl))) +  
  geom_point(size = 2.5) +  
  geom_smooth(method = 'lm', se = FALSE, linewidth = 0.8) +  
  scale_color_manual(values = c('#3AAFA9', '#E8A87C', '#A06CD5')) +  
  labs(title = NULL, x = 'Weight (1000 lbs)', y = 'Miles per gallon') +  
  theme_classic(base_size = 14) +  
  theme(legend.position = 'top',  
        legend.title = element_blank())  
  
ggsave('figure1.pdf', width = 6, height = 4)
```

一张能投稿的图 = 干净的主题 + 明确的坐标轴 + 矢量输出

- theme_classic() 是投稿常用起点
- base_size 一次性控制所有字号比例
- 期刊通常不要标题，标题写在图注里
- ggsave 指定尺寸，PDF 是矢量格式
- 宽度 6 英寸约等于单栏图的标准宽度

练习 4：把图拼起来

目标：体会「位置拼」与「数据拼」的区别

```
library(patchwork)

p1 <- ggplot(mtcars, aes(wt, mpg)) + geom_point()
p2 <- ggplot(mtcars, aes(factor(cyl))) + geom_bar()

# 按位置拼：解决「放在哪里」
p1 | p2
p1 / p2

# 想按数据对齐？试试 aplot
# devtools::install_github("YuLab-SMU/aplot")
# 让两张图共享同一套坐标，顺序自动同步
```

patchwork 用 | 和 / 表达左右与上下

位置拼图简单直接，但数据变化后要重排

数据拼图 (aplot) 适合有对应关系的图

思考：你手上的数据，哪两张图应该对齐？

拼图前先问：这两张图之间有数据对应关系吗？

LAB 05

练习 5：用自己的数据画一张

把课件上的代码换成你自己的数据

找一份数据

从你的课题里挑一份不太大的表格，读进 R

先想清楚问题

用一句话写出你想让读者看到什么

选 geom

根据问题选形状，不要凭喜好选

调外观

换主题、改配色、调字号

存下来

ggsave 输出 PDF 与 PNG 各一份

写下来

把代码整理成脚本，加注释，下次能重跑

能用自己的数据复现一遍，这一讲才算真的学会

89 / 96

PITFALLS

新手最常踩的 8 个坑

每一个都能让新手卡半天

aes 位置写错

color = 'red' 在 aes 里会多出一个图例，写死要放外面

+ 放在行首

R 会以为上一行已经结束。+ 必须放在行尾

因子当数值

as.numeric(factor) 拿到的是编号，不是标签对应的值

geom_bar 重复汇总

geom_bar 自己会数数，先 summarize 会画错

忘记 group

画折线图没设 group，所有点会被连成一条线

坐标系丢数据

scale_x_continuous(limits) 会删数据，
缩放要用 coord_cartesian

颜色不分性别

红绿搭配对色盲不友好，用 viridis 或 Set2

图只在屏幕上

一定要 ggsave 出矢量文件，截图放进论文会糊

这些坑踩过一次就够了，关键是要知道坑在哪里

90 / 96

LEARNING PATH

接下来怎么学

从看懂，到改得出，再到画得巧

1 看懂

任何一张 ggplot2 图都能拆成五个部件

2 改得出

拿到别人的代码，能改成自己的数据

3 画得巧

遇到新需求，知道去哪个扩展包里找

4 成作品

一张图能独立讲清一个结论

可视化是手艺，只能靠反复做来长进。以后同一份数据，既要给人看，也要给模型读

91 / 96

RESOURCES

推荐资源

按顺序看，不要一上来就啃书

ggplot2 官方书

ggplot2: Elegant Graphics for Data Analysis
，第三版在线免费

R Graphics Cookbook

Winston Chang 著，按「我要做 X」组织，最实用

ggplot2 扩展画廊

exts.ggplot2.tidyverse.org，找扩展包的第一站

aplot 文档

yulab-smu.top，数据驱动拼图的入门材料

R Graph Gallery

r-graph-gallery.com，按图形类型索引代码

本讲课件配套代码

把课件里的每一段都跑一遍，比读十篇教程有用

资源不在多，在有没有把其中一份从头到尾跑完

92 / 96

GLOSSARY

术语对照

英文术语和中文说法对不上时，看这张表

英文	中文	一句话解释
aesthetics (aes)	映射	把数据列接到视觉通道上
geom	几何对象	图里能看到的形状：点、线、柱
scale	标度	数值如何转换成颜色、大小、位置
coord	坐标系	默认笛卡尔，可换极坐标或固定比例
theme	主题	字体、颜色、留白等外观设置
facet	分面	按分组变量把图切成多个小面板
layer	图层	用 + 叠加的每一个 geom 或统计变换
grob	图形对象	grid 系统里可编辑的绘图单元

术语是入门的门槛，跨过去就轻松了

93 / 96

FULL CIRCLE

现在再看这 120 个数字

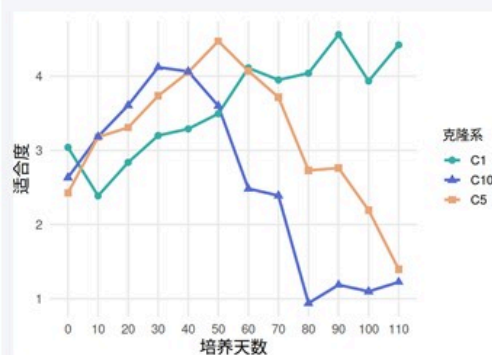
同一批数据，三种画法，回答三个不同的问题

120

个数字，一张表

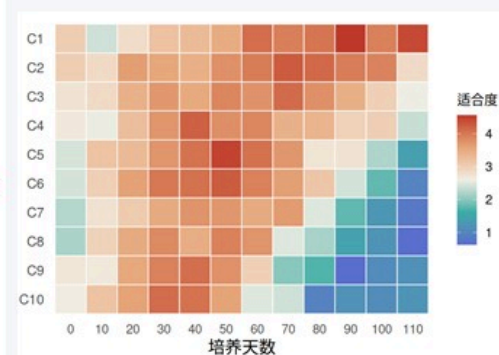
表格：只能查值

第 14 页那 120 个数，看不出趋势



折线：谁升谁降

C1 走高，C5 先升后降，C10 崩落



热图：何时分化

前 50 天同色，第 70 天起裂成两半

图形语法没有多给出一个数字，只是让你多问出三个问题

94 / 96

THINK ABOUT IT

你上一次做的图，如果数据换一批，需要重画多久？

- 如果需要半天以上，说明流程还没有代码化
- 如果只需要改一个文件名，说明你已经掌握了这一讲的核心
- 可视化的终点不是好看，是「可重复」与「可维护」
- 下一次作图前，先问自己：这张图的每一个部件，我能不能说出来？

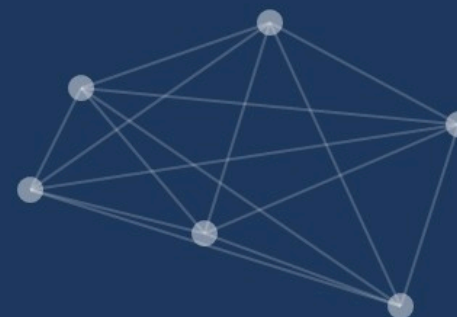
把这个问题记在心里，它会一直提醒你该往哪走

95 / 96

谢谢

Have fun with your data

有问题随时问



96 / 96